

The MagicFrens “Cauldron” Protocol

An Independent, From-Scratch Security Audit
with an Arbitrum Nitro / Orbit L2 Deployment Analysis

Independent review, derived solely from the Solidity sources

2 September 2026

Abstract

The Cauldron is a self-perpetuating token launchpad built on Uniswap v4: a hook meters volume and collects an ETH fee, a registry summons a fixed-supply generation token into a two-position pool (a tradeable *active* band and an out-of-range *reserve* band that backs 1:1 migration and an NFT redemption floor), a governor selects the next brew, and a hook-native perpetuals engine writes real price impact into the same pool. When volume dies, anybody may `relaunch()` and the machine is reborn.

This report is an independent, from-scratch review of all 47 in-scope Solidity files (618 functions across 107 declared types (476 external/public, 142 internal/private)). It was produced without reference to any prior audit, report, exploit write-up or design document in the repository; every claim below is re-derived from the sources and, wherever possible, demonstrated on a live Uniswap v4 fork.

We report **22 findings**: 6 High, 8 Medium and 8 Low/Informational. Six of them are demonstrated by executable proofs of concept. The most consequential are: a complete bypass of the protocol’s only revenue mechanism via exact-output sells; a liquidation oracle that freezes whenever the perpetuals book is empty, so the next position opened is liquidated inside its own opening transaction; an unbounded governor scan that lets a single NFT holder permanently brick the rebirth for a few hundredths of an ETH; a break-glass function that permanently strands an entire generation’s liquidity and bricks `relaunch()`; a governance handoff that irrecoverably burns every economic setter; and — specific to the stated deployment target — a death clock denominated in `block.number`, which on an Arbitrum Orbit chain lets any passer-by permanently destroy a healthy, liquid token roughly an hour after launch.

All six High findings and three of the Medium findings have been **fixed** in this engagement, and each proof of concept has been converted into a regression test that asserts the exploit is now blocked. The suite stands at **302 passing tests across 57 suites**, including handler-based stateful invariant runs against live Uniswap v4.

Contents

1	Scope and Methodology	3
1.1	Scope	3
1.2	Independence	3
1.3	Method	3
2	System Architecture	3
2.1	Value topology	3
2.2	Interaction graph	4
2.3	The lifecycle	4
2.4	The facet split	4
3	Threat Model	5
4	L2 (Arbitrum Nitro / Orbit) Deployment Considerations	5
4.1	The governing fact: <code>block.number</code> is not an L2 block counter	5
4.2	<code>block.timestamp</code>	5
4.3	<code>block.prevrandao</code>	5
4.4	<code>blockhash</code>	6
4.5	<code>gasleft()</code> and the 63/64 rule	6
4.6	<code>tx.origin</code>	6
4.7	Sequencer, mempool and reorgs	6
5	Function Inventory	6

6	Invariant Register	15
7	Findings Register	18
7.1	Z-01 — Exact-output sells pay zero hook fee and still mint crystal credit	18
7.2	Z-02 — The liquidation mark freezes whenever the perpetuals book is empty	19
7.3	Z-03 — Unbounded governor scan lets one NFT holder brick the rebirth	19
7.4	Z-04 — <code>rescueSeeder</code> permanently strands the active book and bricks rebirth	20
7.5	Z-05 — Block-denominated death clock destroys live tokens on the deployment target	20
7.6	Z-06 — Registry ownership is burned into the presale contract at ignition	21
7.7	Z-07 — <code>relaunch()</code> gas is unbounded in practice	21
7.8	Z-08 — <code>setClaimGate</code> froze all migration instantly, un-announced and un-vetoable	22
7.9	Z-09 — Crystal credit accrued on retired generations’ pools	22
7.10	Z-10 — Depth-derived risk caps are flash-manipulable	23
7.11	Z-11 — Queued PLV exits are senior to live shares	23
7.12	Z-12 — Reserve solvency degrades silently at the legacy clamp	24
7.13	Z-13 — The anti-snipe window remains block-denominated	24
7.14	Z-14 — <code>blockhash</code> seed expiry stalls the gacha queue on a fast parent chain	24
7.15	Z-15 — <code>MagicFrensPeg</code> mints for free and its randomness is grindable	25
7.16	Z-16 — Presale refunds pay the original payer regardless of NFT ownership	25
7.17	Z-17 — <code>legacyBuyStep</code> settles its intended amount, not the actual delta	25
7.18	Z-18 — Un-added remainders strand in the registry	26
7.19	Z-19 — <code>liquidityForTokenOut</code> overflows on a degenerate band	26
7.20	Z-20 — In-swap gas reserves are unaudited magic numbers	26
7.21	Z-21 — <code>tx.origin</code> attribution under L2 aliasing	26
7.22	Z-22 — Reserve backing drifts a few tens of wei below its accounting	27
8	Failed Attacks and Safety Arguments	27
9	Third Pass — Final Red-Team (Lead Closure + New Vector)	28
9.1	Lead ledger	28
9.2	Y-01 — Reserve-ceiling breach voids the exit guarantee above $\sim 69\times$	29
9.3	Y-02 — Depth-derived risk caps are flash-manipulable (resolves Z-10)	29
9.4	Fixes applied in this pass	30
10	Remediation Summary	30
10.1	EIP-170 impact	30
10.2	Storage-layout safety	30
11	Recommendations Before Mainnet	31
A	Test Suite and Results	31
A.1	Commands	31
A.2	Result	31
B	Harness Defects Discovered During This Work	32

1 Scope and Methodology

1.1 Scope

All Solidity under `contracts/solidity/` excluding `lib/`, `out/`, `cache/` and `vendor/`. Table 1 lists the reviewed units by subsystem. Deployment scripts under `deploy/` were reviewed as configuration, since several findings are wiring defects rather than code defects.

Subsystem	Units	LoC
Core lifecycle	CauldronRegistry, CauldronBase, RedemptionExt, CauldronToken	1,871
V4 hook	CauldronHook	1,803
Pool plumbing	PoolOps, ReserveLib, SeedLib, CauldronSeeder	1,472
Perpetuals	PerpEngine, PerpVault, PerpStakerOracle	1,709
Collections & floors	CauldronCollection, CauldronVault, CollectionLedger, CauldronFactory	689
Genesis & dividends	MiFrensGenesis, MiFrensDividend	861
Governance	CauldronGovernor, MigrationVesting	619
Periphery	CauldronGachaRouter, LaunchSniper, RoyaltyRouter, DefaultFeeRouter	531
Interfaces & policies	ICauldron, IPolicies, ISeeder, IDeathChecker, ...	218
Out-of-build legacy	MagicFrensPeg, MagicFrensPresale	648
Rendering	FrenRenderer, TraitStorage, SSTORE2	536

Table 1: In-scope units by subsystem.

1.2 Independence

This review was conducted under a hard independence constraint: no pre-existing audit report, exploit write-up, design document, runbook or findings note in the repository was read. Where the source contains inline references to earlier findings (for example “audit A-01”, “audit H-03”), those comments were treated as *claims by the authors to be verified*, not as inputs. Every finding below carries its own derivation and, where the code cites a prior fix, we state independently whether that fix holds.

Two mechanical exceptions were unavoidable and are recorded for transparency: pre-existing test files were edited to keep the suite compiling after the remediation (a two-line idiom substitution applied by script), and file names in those directories appear in build output. No assertion or narrative from them informed any finding.

1.3 Method

The system was read end to end before any finding was written, and modelled as a whole: the cross-contract call graph (§2), the delegatecall facet sharing the registry’s storage, the Uniswap v4 unlock/settle/take flows, the token lifecycle, the reserve accounting, and the perpetuals risk engine. Analysis then proceeded in two passes:

Pass A — coverage

A complete inventory of externally reachable functions with their access gates (§5), and an explicit invariant register with a per-invariant argument for or against (§6).

Pass B — adversarial

For each custody boundary, an attempt to construct a concrete exploit; where none exists, an explicit safety argument (§8). Confirmed exploits were reduced to executable proofs of concept.

Testing ran against a *pinned* Ethereum-Sepolia fork driving the real `PoolManager`, `PositionManager` and canonical `Permit2`. Appendix B records two harness defects discovered during this work that silently invalidate time-dependent tests on this profile; both had to be fixed before several findings could be observed at all.

2 System Architecture

2.1 Value topology

Three custody domains hold essentially all protocol value:

1. **The pool positions.** A generation’s entire 777,000,000-token supply lives inside Uniswap v4 as two positions owned by the registry: a full-range *active* position, and a single-sided *reserve* position parked below the launch tick (pure `token1` until the token appreciates roughly 69×). The reserve is the sole backing for 1:1 migration, the genesis redemption floor and the collection legacy floors — three claimants on one pot.
2. **The hook’s ETH.** `relaunchETH`, `legacyBuffer` and `proposerOwed` are three independently tracked pots inside `CauldronHook`, funded by the swap fee and spent on the next generation’s liquidity, on token buybacks, and on proposer rewards.
3. **The perpetuals vault.** `PerpEngine` holds depositor ETH (`plv`), lent token inventory (`plvToken`), an insurance buffer, and a segregated short-side yield pot.

2.2 Interaction graph

Arrows denote a call; a dagger (†) marks a `delegatecall` that executes in the caller’s storage and custody.

From	To	Purpose
<code>MiFrensGenesis</code>	<code>CauldronRegistry</code>	<code>finalize()</code> → <code>summon()</code> (ignition)
<code>CauldronRegistry</code>	<code>PoolOps</code> †	pool creation, seeding, reserve claims, migration
<code>CauldronRegistry</code>	<code>RedemptionExt</code> †	OG redeem / buy-back / donate / materialise
<code>CauldronRegistry</code>	<code>CauldronHook</code>	<code>setLiveKey</code> , <code>setCollection</code> , <code>releaseRelaunchETH</code>
<code>CauldronRegistry</code>	<code>CauldronSeeder</code>	<code>startSeed</code> , <code>withdrawAll</code> , <code>rescue</code>
<code>CauldronRegistry</code>	<code>CauldronGovernor</code>	<code>hasProposals</code> , <code>winner</code> , <code>markConsumed</code>
<code>CauldronRegistry</code>	<code>CauldronFactory</code>	per-brew collection + vault deploy
<code>CauldronRegistry</code>	<code>CollectionLedger</code>	legacy floor cap table
<code>PoolManager</code>	<code>CauldronHook</code>	<code>beforeSwap</code> / <code>afterSwap</code> / <code>afterInitialize</code>
<code>CauldronHook</code>	<code>PerpEngine</code>	<code>sweepLiquidations</code> (in-swap, gas-bounded)
<code>CauldronHook</code>	<code>CauldronSeeder</code>	<code>pokeInSwap</code> (in-swap streaming)
<code>CauldronHook</code>	<code>CauldronHook</code>	<code>legacyBuyStep</code> , <code>nativeGachaStep</code> (self-calls)
<code>CauldronHook</code>	<code>MiFrensDividend</code>	guild fee slice + anti-snipe surtax
<code>PerpEngine</code>	<code>PoolManager</code>	real settlement swaps (open / close / liquidate)
<code>PerpEngine</code>	<code>CauldronRegistry</code>	<code>claimByBurnUpTo</code> (inventory migration)
<code>PerpVault</code>	<code>PerpEngine</code>	deposit / withdraw working capital
<code>MigrationVesting</code>	<code>CauldronRegistry</code>	<code>claimByBurn</code> behind the vesting gate

2.3 The lifecycle

`summon()` deploys a fixed-supply token (plain `CREATE`), initialises the pool, seeds the two positions — either atomically with a “green-candle” exact-output buy, or progressively by handing the active tranche to `CauldronSeeder`, which streams it in as a side effect of organic trading — and deploys the brew’s NFT collection. Trading accrues fees, crystal credit and volume. When 24-hour volume falls below the threshold the pool reads *dead*, and `relaunch()` force-closes perpetuals, tears down the LP, burns the recovered tokens, crystallises the dying collection’s floor, launches the governor’s winner, and re-seeds. Old holders migrate 1:1 by burning.

2.4 The facet split

`CauldronRegistry` sits 945 bytes under EIP-170 only because the OG-redemption bodies were extracted into `RedemptionExt`, which the registry reaches by `delegatecall` with the original calldata. Both derive from `CauldronBase` and add *no* state, so their layouts are identical by construction — the crux of the design. We verified this independently (§6, INV-F); note in particular that `poolManager`, `positionManager` and `hook` are storage variables rather than immutables precisely because immutables resolve against the executing contract’s code and would read as zero inside the facet.

3 Threat Model

Actor	Capability	Assumed goal
Anonymous trader	Any swap, in any of the four (exact-in exact-out)×(buy sell) quadrants, directly against the <code>PoolManager</code>	Avoid the fee; farm crystal credit; extract from the hook’s own buy-back
MiFren holder (1 NFT)	<code>propose</code> , <code>vote</code> , <code>redeem0gFren</code> , <code>castSpell</code>	Brick or capture the rebirth
Perp counterparty	<code>openLong/openShort/close/liquidate</code> , plus arbitrary pool liquidity	Push loss onto the PLV; farm liquidations; evade liquidation
Keeper / MEV bot	<code>relaunch</code> , <code>poke</code> , <code>resolveTickets</code> , <code>forceCloseDead</code> , transaction ordering	Destroy a live token; extract from in-swap side effects
Liquidity provider	Add/remove liquidity in any pool naming the hook	Manipulate the depth-derived risk limits; re-provision a retired pool
Governance (timelock)	Every <code>onlyEmergency</code> and <code>onlyOwner</code> path	Modelled as <i>honest but fallible</i> ; we test whether a mistake is recoverable
Guardian	<code>vetoEmergency</code> only	Modelled as hostile; can only cancel

Trust assumptions taken as given: Uniswap v4 core and periphery are correct; OpenZeppelin is correct; the sequencer orders honestly (we *do* consider its first-come-first-served, mempool-less ordering when assessing front-running).

4 L2 (Arbitrum Nitro / Orbit) Deployment Considerations

The protocol is written against Ethereum’s execution semantics but is destined for the “Robinhood” chain, an Arbitrum Nitro/Orbit L2. The test suite runs against an L1-like Sepolia fork and therefore cannot reproduce those semantics; the analysis below is derived from Nitro’s documented behaviour and, where it could be modelled, exercised by advancing `block.number` and `block.timestamp` *independently*.

4.1 The governing fact: `block.number` is not an L2 block counter

On Arbitrum Nitro, `block.number` reports the *parent chain’s* block number, not the count of L2 blocks; it advances in jumps, and many L2 blocks share one value. (`ArbSys.arbBlockNumber()` is the L2 counter.) The wall-clock meaning of any block-denominated window is therefore a property of the settlement layer:

Constant (original)	Blocks	Orbit L2 (≈ 12 s parent)	Orbit L3 (≈ 0.25 s parent)
<code>BLOCKS_PER_HOUR</code>	300	1 h	75 s
<code>BLOCKS_PER_DAY</code>	7 200	24 h	30 min
<code>snipeWindowBlocks</code>	30	≈ 6 min	≈ 7.5 s
<code>blockhash</code> horizon	256	≈ 51 min	≈ 64 s

This single fact drives finding Z-05 (**High**) and residuals Z-13 and Z-14. Because the two volume constants were declared `constant`, there was no setter and no migration path, and `CauldronHook` had 58 bytes of EIP-170 headroom — one could not have been added. The remediation re-denominates the volume window in wall-clock seconds, which carries the intended meaning identically on L1, on an Orbit L2 and on an Orbit L3.

4.2 `block.timestamp`

Nitro timestamps are the sequencer’s clock at one-second granularity, non-decreasing and bounded relative to L1. Every timestamp-keyed mechanism in the protocol therefore ports cleanly: the emergency timelock, `minLifetime`, the governor’s `VOTING_PERIOD`, the vesting window, the TWAP window, `OBS_INTERVAL`, funding Δt , and the perpetuals warm-up. The engine’s per-block liquidation throttle is already keyed on `block.timestamp` with a correct in-source justification. One consequence deserves recording: because that cap resets *every second* rather than every 12 seconds, its *per-minute* bound is roughly 12× looser than the L1 reading of the same parameter, even though the atomic-cascade guarantee is unchanged (Z-10).

4.3 `block.prevrandao`

Constant on Arbitrum/Orbit. The protocol correctly does not rely on it for any live randomness: the crystal gacha and both NFT reveals use commit-reveal over `blockhash`, and the anti-snipe jitter folds in the live

pool tick precisely because `prevrandao` is useless there. The sole remaining use is `MagicFrensPeg._random`, an out-of-build legacy contract (Z-15).

4.4 blockhash

Nitro documents `blockhash(x)` as returning a *cryptographically insecure, pseudo-random* value for x in `[block.number - 256, block.number)`, zero outside it, and explicitly *not* the L1 block hash. Two consequences:

1. Every commit in one parent-block window shares a seed. Per-ticket variation comes from `keccak(bh, player, batch, index)`, so outcomes remain distinct, but the entropy source is one documented-insecure value. We record this as *Suspected — requires validation on the target chain*: whether the value is predictable *before* the window closes depends on the ArbOS derivation, which we could not verify from the sources in scope.
2. The 256-block horizon collapses to roughly a minute on a fast parent chain, so seed expiry stops being exceptional. The expiry path re-anchors and **breaks**, which head-of-line blocks the whole FIFO queue and hands the batch’s owner a fresh roll (Z-14, demonstrated).

4.5 gasleft() and the 63/64 rule

Nitro deducts the L1 data-posting cost up front and then meters execution on the ordinary EVM schedule, so the hook’s *gas-capped* sub-calls port safely: each forwards `gasleft()` — R and keeps exactly R , so an out-of-gas child consumes precisely its budget. The dangerous cases are the calls that are *not* capped — `hook.forceClosePerps()` and `hook.resolveTickets(300)` inside `relaunch()` — where the 63/64 rule leaves the parent only 1/64 of its gas and the rebirth dies with it (Z-07).

4.6 tx.origin

Used for crystal attribution on untagged swaps and for liquidator credit. For ordinary L2 transactions this behaves as on L1. For L1→L2 retryables the origin is the *aliased* L1 address, so a keeper reward can be credited to an address controlled only from L1 (Z-21). The design note in the source correctly identifies the ERC-4337 caveat.

4.7 Sequencer, mempool and reorgs

A single sequencer with no public mempool removes classical mempool sandwiching, which materially weakens the front-running threat against the anti-snipe window and the TWAP mark. It does *not* remove ordering games where the attacker can *cause* the event they wish to bracket — notably the legacy buy-back, whose trigger condition (`legacyBuffer >= legacyThreshold`) is publicly computable and can be tripped by the attacker’s own swap in the same transaction (Z-17, bounded by the buffer size).

5 Function Inventory

The parser reports 618 functions across 107 declared types (476 external/public, 142 internal/private). Table 2 lists every externally reachable function with its visibility, mutability and access gate; the gate column is the decisive one for Pass A, since it is the complete authorisation surface of the protocol.

Table 2: Complete inventory of externally reachable functions in scope.

Contract	Function	#	Vis.	Mutability	Access gate
IPerpEngineLiq	<code>liquidateInSwap</code>	2	external	nonpayable	—
	<code>liquidateManyInSwap</code>	2	external	nonpayable	—
	<code>sweepLiquidations</code>	1	external	nonpayable	—
ICollectionLiquidator	<code>setLiquidatorMinter</code>	1	external	nonpayable	—
ILegacyNote	<code>noteLegacyBuy</code>	1	external	nonpayable	—
IPerpForceClose	<code>forceCloseAllDead</code>	0	external	nonpayable	—
IPerpFeeCredit	<code>creditPerpFee</code>	0	external	payable	—
	<code>creditPerpFeeToken</code>	0	external	payable	—
ISeederInSwap	<code>pokeInSwap</code>	0	external	nonpayable	—
CauldronHook	<code>getHookPermissions</code>	0	public	pure	—

continued on next page

continued from previous page

Contract	Function	#	Vis.	Mutability	Access gate
CauldronRegistry	legacyBuyStep	1	external	nonpayable	—
	fundLegacyBuffer	0	external	payable	—
	sweepLegacyReserve	2	external	nonpayable	—
	snipeSurtaxBps	1	public	view	—
	setSnipeParams	2	external	nonpayable	onlyOwner
	getVolume24h	1	public	view	—
	isDead	1	external	view	—
	setDefaultTaxBps	1	external	nonpayable	onlyOwner
	releaseRelaunchETH	0	external	nonpayable	—
	setDeathThreshold	1	external	nonpayable	onlyOwner
	forceClosePerps	0	external	nonpayable	—
	setLiveKey	1	external	nonpayable	—
	liveKey	0	external	view	—
	setLegacyBuyback	3	external	nonpayable	—
	setDeathChecker	1	external	nonpayable	—
	setPolicies	3	external	nonpayable	—
	setFeeRouter	1	external	nonpayable	—
	setNftContract	1	external	nonpayable	onlyOwner
	setTreasury	1	external	nonpayable	onlyOwner
	setRegistry	1	external	nonpayable	onlyOwner
	proposeRegistryOverride	1	external	nonpayable	onlyOwner
	cancelRegistryOverride	0	external	nonpayable	onlyOwner
	executeRegistryOverride	0	external	nonpayable	onlyOwner
	setCollection	1	external	nonpayable	—
	setVault	1	external	nonpayable	—
	setQuest	1	external	nonpayable	—
	setSeeder	1	external	nonpayable	—
	setPerpEngine	1	external	nonpayable	—
	setFloorBps	1	external	nonpayable	onlyOwner
	setGuild	1	external	nonpayable	—
	setGuildBps	1	external	nonpayable	onlyOwner
	setActiveProposer	1	external	nonpayable	—
	setProposerBps	1	external	nonpayable	onlyOwner
	claimProposerFees	0	external	nonpayable	nonReentrant
	setNftCurve	2	external	nonpayable	onlyOwner
	setCreditUntaggedSwaps	1	external	nonpayable	onlyOwner
	setNftCurveFrom	1	external	nonpayable	—
	nftPriceAt	1	public	view	—
	creditOf	1	external	view	—
	crystalsReady	1	public	view	—
	costOfNextCrystals	1	public	view	—
	oddsForPlay	1	public	view	—
	outstandingTickets	0	external	view	—
	mintedOut	0	external	view	—
	progress	1	external	view	—
	commitCrystals	3	external	nonpayable	nonReentrant
	resolveTickets	1	public	nonpayable	nonReentrant
	nativeGachaStep	2	external	nonpayable	nonReentrant
	setOpener	2	external	nonpayable	—
	setTaxExempt	2	external	nonpayable	—
	setOddsParams	2	external	nonpayable	onlyOwner
	setMaxOdds	1	external	nonpayable	onlyOwner
	setWeights	2	external	nonpayable	onlyOwner
	setRedemptionExt	1	external	nonpayable	onlyOwner
	setReserveCeiling	1	external	nonpayable	onlyOwner
	setSeeder	1	external	nonpayable	onlyOwner

continued on next page

continued from previous page

Contract	Function	#	Vis.	Mutability	Access gate
CauldronToken IFrenRenderer MagicFrensPeg	rescueSeeder	0	external	nonpayable	onlyEmergency, time-locked, nonReentrant
	setSeedWindow	1	external	nonpayable	onlyOwner
	enchantFee	0	external	view	—
	setIgniter	1	external	nonpayable	onlyOwner
	armEmergency	0	external	nonpayable	onlyEmergency
	setGuardian	1	external	nonpayable	—
	vetoEmergency	0	external	nonpayable	—
	setRedemptionPaused	1	external	nonpayable	onlyEmergency
	setEnchantFeeMult	1	external	nonpayable	onlyEmergency
	emergencyWithdrawLP	1	external	nonpayable	onlyEmergency, time-locked, nonReentrant
	emergencySweep	1	external	nonpayable	onlyEmergency, time-locked, nonReentrant
	setSuccessor	1	external	nonpayable	onlyEmergency
	setClaimGate	1	external	nonpayable	onlyEmergency
	migrateToSuccessor	0	external	nonpayable	onlyEmergency, time-locked, nonReentrant
	setGovernor	1	external	nonpayable	onlyOwner
	setMinLifetime	1	external	nonpayable	onlyEmergency
	setFactory	1	external	nonpayable	onlyOwner
	setNftMaxSupply	1	external	nonpayable	onlyOwner
	setRoyalty	2	external	nonpayable	onlyOwner
	setGenesisMetadata	3	external	nonpayable	onlyOwner
	setGenesisBonus	3	external	nonpayable	onlyOwner
	setAirdropReserve	2	external	nonpayable	onlyOwner
	setPrimeFunder	1	external	nonpayable	onlyOwner
	fundPrimeBuy	0	external	payable	—
	sweepPrimeBuy	0	external	nonpayable	—
	summon	0	external	payable	nonReentrant
	relaunch	0	external	nonpayable	nonReentrant
	claimByBurn	2	external	nonpayable	—
	claimByBurnUpTo	2	external	nonpayable	—
	enableAutoMigrate	0	external	payable	—
	autoMigrateBatch	2	external	nonpayable	nonReentrant
	redeemOgFren	1	external	nonpayable	—
	buyTreasuryOgFren	1	external	nonpayable	—
	donateToReserve	1	external	nonpayable	—
	materializeLegacyReserve	0	external	nonpayable	—
	setCollectionLedger	1	external	nonpayable	onlyOwner
	recycleCollectionNFT	2	external	nonpayable	nonReentrant
	buyCollectionNFT	2	external	nonpayable	nonReentrant
	unlockCallback	1	external	nonpayable	—
	hasClaimed	2	external	view	—
	burn	2	external	nonpayable	onlyRegistry
	tokenURI	5	external	view	—
	buyFren	0	external	payable	nonReentrant
	sellFren	0	external	nonpayable	nonReentrant
	commitFren	0	external	nonpayable	nonReentrant
	transferCommittedFren	2	external	nonpayable	—
	getTraits	1	external	view	—
	hasFren	1	external	view	—
	getFrenId	1	external	view	—
	setTreasury	1	external	nonpayable	onlyOwner
	transferFrom	3	public	nonpayable	override(ERC20, IERC721)

continued on next page

continued from previous page

Contract	Function	#	Vis.	Mutability	Access gate
MagicFrensPresale	supportsInterface	1	public	view	override(ERC721, ERC721Enumerable)
	setRenderer	1	external	nonpayable	onlyOwner
	tokenURI	1	public	view	—
	contribute	0	external	payable	nonReentrant
	claimTokens	0	external	nonpayable	nonReentrant
	endPresale	0	external	nonpayable	onlyOwner
	setTokenAddress	1	external	nonpayable	onlyOwner
	setTreasury	1	external	nonpayable	onlyOwner
	withdraw	0	external	nonpayable	onlyOwner
	togglePause	0	external	nonpayable	onlyOwner
	addToWhitelist	1	external	nonpayable	onlyOwner
	toggleWhitelistOnly	0	external	nonpayable	onlyOwner
ICauldronFactory	getContribution	1	external	view	—
	getPresaleStats	0	external	view	—
IMiFrensContinuable	deployBrew	1	external	nonpayable	—
	deployVault	3	external	nonpayable	—
	setMinter	1	external	nonpayable	—
	setVault	1	external	nonpayable	—
	totalMinted	0	external	view	—
	custodyTransfer	3	external	nonpayable	—
IVaultClose	everMoved	1	external	view	—
	close	0	external	nonpayable	—
IPerpSync	syncGeneration	0	external	nonpayable	—
ICollectionLedger	totalEntitled	0	external	view	—
IPositionManager	modifyLiquidities	2	external	payable	—
	nextTokenId	0	external	view	—
CauldronBase	getPositionLiquidity	1	external	view	—
	floorPerFren	0	public	view	—
CauldronCollection	getTransferValidator	0	external	view	—
	getTransferValidationFunction	1	external	pure	—
	setTransferValidator	1	external	nonpayable	—
	supportsInterface	1	public	view	override(ERC721, ERC2981)
	mint	1	external	nonpayable	—
	reveal	1	external	nonpayable	—
	setVault	1	external	nonpayable	—
	setRoyalty	2	external	nonpayable	—
	setLiquidatorMinter	1	external	nonpayable	—
	setMetadata	3	external	nonpayable	—
	setLiquidatorURI	1	external	nonpayable	—
	mintLiquidator	1	external	nonpayable	—
	liquidatorTrait	1	external	view	—
	burnFromVault	1	external	nonpayable	—
	custodyTransfer	3	external	nonpayable	—
	tokenURI	1	public	view	—
	setRarityOdds	1	external	nonpayable	—
	deployBrew	1	external	nonpayable	—
	deployVault	3	external	nonpayable	—
	commitCrystals	3	external	nonpayable	—
	resolveTickets	1	external	nonpayable	—
	crystalsReady	1	external	view	—
	costOfNextCrystals	1	external	view	—
	buyWeightBps	0	external	view	—
IRegistryCurrent	currentToken	0	external	view	—
CauldronGachaRouter	play	4	external	payable	nonReentrant

continued on next page

continued from previous page

Contract	Function	#	Vis.	Mutability	Access gate
CauldronGovernor	playLiq	5	external	payable	nonReentrant
	openReady	1	external	nonpayable	nonReentrant
	playChurn	2	external	payable	nonReentrant
	unlockCallback	1	external	nonpayable	—
	rescueETH	2	external	nonpayable	onlyOwner
	setRegistry	1	external	nonpayable	onlyOwner
	propose	9	external	nonpayable	—
	displayName	1	external	view	—
	vote	1	external	nonpayable	—
	winner	0	external	view	—
	hasProposals	0	external	view	—
CauldronSeeder	markConsumed	1	external	nonpayable	—
	getProposal	1	external	view	—
	startSeed	1	external	payable	onlyRegistry, lock
	poke	0	external	nonpayable	lock
	pokeInSwap	0	external	nonpayable	lock
	unlockCallback	1	external	nonpayable	—
	withdrawAll	1	external	nonpayable	onlyRegistry, lock
	rescue	1	external	nonpayable	onlyRegistry, lock
	deployedWad	0	external	view	—
	rangeCount	0	external	view	—
	isComplete	0	external	view	—
IBurnableCollection	ownerOf	1	external	view	—
	totalMinted	0	external	view	—
	burnFromVault	1	external	nonpayable	—
CauldronVault	outstanding	0	public	view	—
	floorPerNFT	0	public	view	—
	redeem	1	external	nonpayable	nonReentrant
CollectionLedger	close	0	external	nonpayable	nonReentrant
	outstanding	2	public	view	—
	floorPerNFT	2	public	view	—
	credit	2	external	nonpayable	onlyRegistry
	redeem	2	external	nonpayable	onlyRegistry
DefaultFeeRouter	buyback	3	external	nonpayable	onlyRegistry
	crystallize	3	external	nonpayable	onlyRegistry
	route	5	external	pure	—
ICollectionRenderer	tokenURI	1	external	view	—
ICauldronGovernor	winner	0	external	view	—
	markConsumed	1	external	nonpayable	—
	hasProposals	0	external	view	—
ICauldronCollection	mint	1	external	nonpayable	—
	totalMinted	0	external	view	—
	maxSupply	0	external	view	—
ITransferValidator	validateTransfer	4	external	view	—
ICreatorToken	getTransferValidator	0	external	view	—
	getTransferValidationFunction	1	external	view	—
IDeathChecker	setTransferValidator	1	external	nonpayable	—
	isDead	3	external	view	—
ILiquidatorMintable	mintLiquidator	1	external	nonpayable	—
ISurtaxPolicy	surtaxBps	4	external	view	—
IOddsPolicy	oddsBps	3	external	view	—
ICurvePolicy	priceAt	3	external	view	—
IFeeRouter	route	5	external	view	—
ISeeder	startSeed	1	external	payable	—
	poke	0	external	nonpayable	—
	withdrawAll	1	external	nonpayable	—

continued on next page

continued from previous page

Contract	Function	#	Vis.	Mutability	Access gate
IMiFrensGenesisFinalize	rescue	1	external	nonpayable	—
	isComplete	0	external	view	—
	seeding	0	external	view	—
	finalize	0	external	nonpayable	—
	soldOut	0	external	view	—
IRegistryCurrent	currentToken	0	external	view	—
IGachaPlay	play	4	external	payable	—
LaunchSniper	launch	6	external	payable	onlyOwner
	sweep	1	external	nonpayable	onlyOwner
IMiFrensShares	ownerOf	1	external	view	—
	GENESIS_SUPPLY	0	external	view	—
	MAX_SUPPLY	0	external	view	—
	everMoved	1	external	view	—
IReserveRegistry	enchantFee	0	external	view	—
	currentToken	0	external	view	—
	donateToReserve	1	external	nonpayable	—
MiFrensDividend	setRegistry	1	external	nonpayable	—
	pending	1	public	view	—
	isEnchanted	1	external	view	—
	castSpell	1	external	nonpayable	—
	castMany	1	external	nonpayable	—
	onMiFrenTransfer	2	external	nonpayable	—
	claim	1	public	nonpayable	nonReentrant
	claimMany	1	external	nonpayable	nonReentrant
	withdrawOwed	0	external	nonpayable	nonReentrant
	summon	0	external	payable	—
IRegistrySummon	summoned	0	external	view	—
	onMiFrenTransfer	2	external	nonpayable	—
MiFrensGenesis	setRegistry	1	external	nonpayable	—
	mint	1	external	payable	nonReentrant
	cancelPresale	0	external	nonpayable	—
	refund	0	external	nonpayable	nonReentrant
	totalMinted	0	external	view	—
	maxSupply	0	external	view	—
	setMinter	1	external	nonpayable	onlyDeployerOrRegistry
	setVault	1	external	nonpayable	onlyDeployerOrRegistry
	setDividend	1	external	nonpayable	onlyDeployerOrRegistry
	setLiquidatorMinter	1	external	nonpayable	—
	setLiquidatorURI	1	external	nonpayable	—
	mintLiquidator	1	external	nonpayable	—
	liquidatorTrait	1	external	view	—
	custodyTransfer	3	external	nonpayable	—
	setFinalizer	1	external	nonpayable	—
	setMetadata	3	external	nonpayable	—
	setRarityOdds	1	external	nonpayable	—
	setRoyalty	2	external	nonpayable	—
	getTransferValidator	0	external	view	—
	getTransferValidationFunction	1	external	pure	—
	setTransferValidator	1	external	nonpayable	—
	mint	1	external	nonpayable	—
	reveal	1	external	nonpayable	—
	burnFromVault	1	external	nonpayable	—
	finalize	0	external	nonpayable	nonReentrant
	soldOut	0	external	view	—
	remaining	0	external	view	—
	isGenesis	1	public	view	—

continued on next page

continued from previous page

Contract	Function	#	Vis.	Mutability	Access gate
	ogTrait	1	external	view	–
	tokenURI	1	public	view	–
	supportsInterface	1	public	view	override(ERC721, ERC2981)
IVestingRegistry	currentGeneration	0	external	view	–
	generationToken	1	external	view	–
	claimByBurn	2	external	nonpayable	–
IStakerOracle	isInstant	1	external	view	–
MigrationVesting	startVest	2	external	nonpayable	nonReentrant
	vestBatch	2	external	nonpayable	nonReentrant
	claim	0	external	nonpayable	nonReentrant
	claimFor	1	external	nonpayable	nonReentrant
	claimable	1	external	view	–
	locked	1	external	view	–
	grantCount	1	external	view	–
	grantAt	2	external	view	–
	setVestWindow	1	external	nonpayable	onlyOwner
	setStakerOracle	1	external	nonpayable	onlyOwner
IPerpRegistry	currentToken	0	external	view	–
	currentGeneration	0	external	view	–
	lastSummonAt	0	external	view	–
	generationPoolId	1	external	view	–
	generationToken	1	external	view	–
	claimByBurn	2	external	nonpayable	–
	claimByBurnUpTo	2	external	nonpayable	–
IPerpHook	isDead	1	external	view	–
	collection	0	external	view	–
PerpEngine	totalEth	0	public	view	–
	freeEth	0	external	view	–
	totalTokenAssets	0	public	view	–
	freeToken	0	external	view	–
	poke	0	external	nonpayable	–
	twapTick	0	public	view	–
	markSqrtPriceX96	0	public	view	–
	activeEthDepth	0	public	view	–
	maxLeverage	0	public	view	–
	fundingDelta	1	external	view	–
	openLong	3	public	payable	nonReentrant, notNested
	openShort	3	public	payable	nonReentrant, notNested
	close	2	external	nonpayable	nonReentrant, notNested
	liquidate	1	external	nonpayable	nonReentrant, notNested
	sweepLiquidations	1	external	nonpayable	–
	selfSweep	1	external	nonpayable	–
	forceCloseDead	1	external	nonpayable	nonReentrant, notNested
	forceCloseAllDead	0	external	nonpayable	nonReentrant, notNested
	syncGeneration	0	external	nonpayable	nonReentrant, notNested
	isLiquidatable	1	external	view	–
	unlockCallback	1	external	nonpayable	–
	claimPayout	0	external	nonpayable	nonReentrant
	claimLiquidatorBadges	1	external	nonpayable	nonReentrant

continued on next page

continued from previous page

Contract	Function	#	Vis.	Mutability	Access gate
IPerpShares	fundPlv	0	external	payable	onlyOwner, notNested
	fundPlvToken	1	external	nonpayable	onlyOwner, notNested
	fundInsurance	0	external	payable	—
	creditPerpFee	0	external	payable	—
	creditPerpFeeToken	0	external	payable	—
	fundFromVault	0	external	payable	onlyVault
	withdrawPlvTo	2	external	nonpayable	onlyVault, notNested, nonReentrant
	withdrawTokYieldTo	2	external	nonpayable	onlyVault, notNested, nonReentrant
	fundTokenFromVault	1	external	nonpayable	onlyVault
	withdrawPlvTokenTo	2	external	nonpayable	onlyVault, notNested, nonReentrant
	setFees	5	external	nonpayable	onlyOwner
	setRisk	6	external	nonpayable	onlyOwner
	setTwapWindow	1	external	nonpayable	onlyOwner
	setTiers	2	external	nonpayable	onlyOwner
	setRouting	2	external	nonpayable	onlyOwner
	setNftBeneficiary	1	external	nonpayable	onlyOwner
	setGuards	3	external	nonpayable	onlyOwner
	setVault	1	external	nonpayable	onlyOwner
	setVaultSplit	2	external	nonpayable	onlyOwner
	setVaultLimits	2	external	nonpayable	onlyOwner
	setMinCollateral	1	external	nonpayable	onlyOwner
	skimInsurance	2	external	nonpayable	onlyOwner
	positionHealth	1	external	view	—
	ethShareOf	1	external	view	—
	tokShareOf	1	external	view	—
	isInstant	1	external	view	—
	fundFromVault	0	external	payable	—
	withdrawPlvTo	2	external	nonpayable	—
	fundTokenFromVault	1	external	nonpayable	—
	withdrawPlvTokenTo	2	external	nonpayable	—
	totalEth	0	external	view	—
	freeEth	0	external	view	—
	totalTokenAssets	0	external	view	—
	freeToken	0	external	view	—
	tokYieldCumulative	0	external	view	—
	withdrawTokYieldTo	2	external	nonpayable	—
IVaultRegistry	currentToken	0	external	view	—
	assetsEth	0	public	view	—
	assetsTok	0	public	view	—
PerpVault	depositEth	0	external	payable	nonReentrant
	withdrawEth	1	external	nonpayable	nonReentrant
	claimPendingEth	0	external	nonpayable	nonReentrant
	depositToken	1	external	nonpayable	nonReentrant
	claimTokYield	0	external	nonpayable	nonReentrant
	withdrawToken	1	external	nonpayable	nonReentrant
	claimPendingToken	0	external	nonpayable	nonReentrant
	ethPosition	1	external	view	—
	tokenPosition	1	external	view	—
	pendingTokYield	1	external	view	—
IPositionManagerOps	modifyLiquidities	2	external	payable	—
	nextTokenId	0	external	view	—
ILedgerOps	getPositionLiquidity	1	external	view	—
	redeem	2	external	nonpayable	—

continued on next page

continued from previous page

Contract	Function	#	Vis.	Mutability	Access gate
	buyback	3	external	nonpayable	–
	floorPerNFT	2	external	view	–
	credit	2	external	nonpayable	–
	crystallize	3	external	nonpayable	–
	crystallized	1	external	view	–
	totalEntitled	0	external	view	–
IColMinted	totalMinted	0	external	view	–
IVaultRedeemedOps	redeemed	0	external	view	–
	outstanding	0	external	view	–
ICollectionOps	custodyTransfer	3	external	nonpayable	–
	ownerOf	1	external	view	–
ILegacyHookOps	sweepLegacyReserve	2	external	nonpayable	–
	legacyRegistry	0	external	view	–
ICauldronBurn	burn	2	external	nonpayable	–
IAutoFlag	autoMigrate	1	external	view	–
IPermit2Ops	approve	4	external	nonpayable	–
PoolOps	creatureFor	1	external	pure	–
	createAndSeed	10	external	nonpayable	–
	createAndSeedProgressive	11	external	nonpayable	–
	createAndSeedWithBuy	10	external	nonpayable	–
	executeBuy	1	external	nonpayable	–
	primeBuy	4	external	nonpayable	–
	deployToken	4	external	nonpayable	–
	removeAll	4	external	nonpayable	–
	claimFromReserve	7	public	nonpayable	–
	addToReserve	6	public	nonpayable	–
	migrateUpTo	5	external	nonpayable	–
	migrateOne	5	public	nonpayable	–
	autoMigrateBatch	5	external	nonpayable	–
	doLegacyNote	6	public	nonpayable	–
	materializeLegacy	11	external	nonpayable	–
	crystallizeCollection	7	external	nonpayable	–
	recycleCollection	7	external	nonpayable	–
	buyCollection	8	external	nonpayable	–
RedemptionExt	redeemOgFren	1	external	nonpayable	nonReentrant
	buyTreasuryOgFren	1	external	nonpayable	nonReentrant
	donateToReserve	1	external	nonpayable	nonReentrant
	materializeLegacyReserve	0	external	nonpayable	nonReentrant
ILegacyBuffer	fundLegacyBuffer	0	external	payable	–
DeployCauldron	run	0	external	nonpayable	–
IHookExempt	setTaxExempt	2	external	nonpayable	–
IPresaleFinalizer	setFinalizer	1	external	nonpayable	–
DeployLaunchSniper	run	0	external	nonpayable	–
IOwnable	transferOwnership	1	external	nonpayable	–
DeployLaunchpad	run	0	external	nonpayable	–
DeployMagicFrensPeg	run	0	external	nonpayable	–
IRegistryGate	emergencyAdmin	0	external	view	–
	emergencyDelay	0	external	view	–
	claimGate	0	external	view	–
	setClaimGate	1	external	nonpayable	–
IHookPerp	perpEngine	0	external	view	–
IEngineVault	vault	0	external	view	–
DeployMigrationVesting	run	0	external	nonpayable	–
IHookWire	setPerpEngine	1	external	nonpayable	–
	collection	0	external	view	–
ICollLiq	liquidatorMinter	0	external	view	–

continued on next page

continued from previous page

Contract	Function	#	Vis.	Mutability	Access gate
IPerpVaultDeposit	depositEth	0	external	payable	–
IOwnable	transferOwnership	1	external	nonpayable	–
DeployPerp	run	0	external	nonpayable	–
DeployPresale	run	0	external	nonpayable	–
IPegRenderer	setRenderer	1	external	nonpayable	–
DeployRenderer	run	0	external	nonpayable	–
IERC20Min	approve	2	external	nonpayable	–
	balanceOf	1	external	view	–
SellVolume	run	0	external	nonpayable	–
ISeederView	deployedWad	0	external	view	–
	seeding	0	external	view	–
	isComplete	0	external	view	–
IERC20View	balanceOf	1	external	view	–
SnipeBuy	run	0	external	nonpayable	–
IHookView	openNFTs	2	external	nonpayable	–
	openableNFTs	1	external	view	–
	getVolume24h	1	external	view	–
	nftCredit	2	external	view	–
	creditEpoch	0	external	view	–
ICollView	totalMinted	0	external	view	–
	balanceOf	1	external	view	–
SwapVolume	run	0	external	nonpayable	–
INFTContract	getHolderTaxRate	1	external	view	–
	balanceOf	1	external	view	–
FrenRenderer	tokenURI	5	external	view	–
	renderSVG	4	public	view	–
TraitStorage	storePalette	1	external	nonpayable	onlyOwner, notFrozen
	storeTrait	2	public	nonpayable	onlyOwner, notFrozen
	storeTraits	2	external	nonpayable	onlyOwner, notFrozen
	freeze	0	external	nonpayable	onlyOwner
	traitKey	3	public	pure	–
	getTrait	1	external	view	–
	getTraitOrEmpty	3	external	view	–
	getPalette	0	external	view	–
	hasTrait	1	external	view	–

6 Invariant Register

Each invariant is stated, then argued from cited code, then marked with how it is enforced in the test suite.

ID	Statement	Argument	Status
SUP-1	Total supply is conserved: a generation token can never be minted after construction.	CauldronToken mints once in its constructor and exposes only burn (registry-gated). There is no mint . Migration is a <i>transfer</i> out of the reserve position, never a mint.	Holds. Stateful invariant invariant_SUP1 .
MIG-1	Migration is exactly 1:1 and non-duplicable.	claimByBurn burns the caller’s real fromGen balance and pulls the same amount from the reserve; migrateOne reverts if the reserve delivers less than amount - CLAIM_DUST . The right follows the token, so it cannot be claimed twice.	Holds, subject to RES-1.

ID	Statement	Argument	Status
RES-1	The reserve covers migration + genesis + legacy simultaneously.	At relaunch, $newReserve = TOTAL - newActive$ where <code>newActive</code> is reduced by the unclaimed genesis and by <code>totalEntitled</code> . Conservation is exact — see §8, SA-1 — <i>except</i> that the legacy subtraction is silently skipped when $legacy \geq newActive$, and a degenerate zero recovery substitutes an 80% constant.	Breaks in the clamp branch: Medium Z-12.
RES-2	The reserve never delivers more token than was deposited for it.	<code>ReserveLib</code> sizes both directions in liquidity units, which round <i>down</i> ; the round trip is $\lfloor \lfloor x Q/d \rfloor d/Q \rfloor \leq x$.	Holds. Fuzz <code>testFuzz_INV_R</code> (257 runs).
RES-3	Reserve rounding dust stays below <code>CLAIM_DUST</code> (10^{12}), or honest claims revert.	Dust is bounded by $(\sqrt{P_{hi}} - \sqrt{P_{lo}})/Q_{96}$. Over the protocol’s whole launch-price domain this is $O(10^3)$ wei, nine orders below the tolerance.	Holds. Fuzz <code>testFuzz_INV_D</code> .
FLOOR-1	The advertised genesis floor is always backed by the reserve accounting.	<code>floorPerFren</code> = <code>genesisReserveOutstanding/genesisShafab</code> by definition, and <code>redeemOgFren</code> debits before paying.	Holds. Stateful invariant <code>invariant_RES1</code> .
LEDG-1	$totalEntitled = \sum_g entitledTokens[g]$ under any interleaving.	Every mutator adjusts both sides by the same quantity; <code>redeem</code> rounds the payout down so the sum never drifts upward.	Holds. Stateful invariant, 128,000 calls, 0 reverts.
HOOK-1	The hook always holds at least what its books owe.	<code>_takeEthFee</code> takes <code>total</code> and routes exactly <code>total</code> ; failed sends roll into <code>relaunchETH</code> ; <code>legacyBuyStep</code> zeroes the buffer before spending.	Holds. Stateful invariant <code>invariant_HOOK1</code> .
PERP-1	The engine always holds at least <code>plv + insuranceEth + tokYieldEth</code> .	Every debit path (<code>_absorbPlvLoss</code> , <code>_replenishPlv</code> , funding) is saturating, and <code>_payOut</code> credits <code>payoutOwed</code> rather than reverting.	Holds. Stateful invariant <code>invariant_PERP1</code> .
PERP-2	PLV principal is protected behind collateral + insurance.	Long shortfalls replenish <code>plv</code> from insurance first; short overspends are absorbed insurance-first. Only an insurance-exhausting gap socialises onto depositors — which is the documented design.	Holds , but see Z-10 (the risk caps that bound the gap are manipulable) and Z-11 (queued exits are senior).
FUND-1	Funding is a bounded, solvent transfer.	Payers are capped by their own residual and by $\pm \text{maxFundingBps}$ of collateral; receivers draw insurance-first, then <code>plv</code> , never overdrawing.	Holds as an accounting matter; it is <i>not</i> zero-sum (an underwater payer pays less than it owes), and the deficit is charged to insurance by design.
SEED-1	The seeder only ever touches ledger A, never the $69\times$ reserve.	<code>CauldronSeeder</code> custodies only what <code>startSeed</code> transferred; the reserve is a registry-owned <code>PositionManager</code> position the seeder holds no reference to.	Holds — structural.

ID	Statement	Argument	Status
SEED-2	Ledger A is fully deployed and fully recoverable.	The schedule is monotone, bounded and terminates at exactly 10^{18} ; tear-down walks every tracked range.	Broke via rescue (High Z-04); Fixed. Fuzz testFuzz_INV_S.
INV-F	Registry and facet storage layouts are identical.	Both derive from <code>CauldronBase</code> and declare <i>no</i> additional state variables; <code>CauldronRegistry</code> ’s two extra members are <code>immutable</code> and therefore live in code, not storage. The three former <code>immutable</code> s were correctly promoted to storage.	Holds by construction. Preserved by the remediation: the new <code>igniter</code> was appended to <code>CauldronBase</code> , not to a child.
V4-1	No <code>PoolManager</code> delta is ever left unset.	Enforced structurally by v4-core, which reverts <code>unlock</code> on any non-zero delta. Every settlement site was read; all pay the actual delta except <code>legacyBuyStep</code> , which settles its <i>intended</i> amount (Z-17).	Holds for all exercised paths; every successful handler call is a proof for its path.
FEE-1	Every swap on a served pool pays the ETH fee.	Four quadrants; <code>beforeSwap</code> charges exact-input buys, <code>afterSwap</code> charges the two where ETH is the unspecified currency.	Broke for exact-output sells (High Z-01); Fixed.
LIFE-1	Only a genuinely dead pool can be re-launched.	<code>isDead</code> reads the 24-hour volume window; <code>minLifetime</code> adds a grace period.	Broke on the target chain, where the window is $48\times$ too short (High Z-05); Fixed.
LIFE-2	The rebirth can always be executed.	<code>relaunch</code> is permissionless and every sub-call that can fail is <code>try/catch</code> ’d ... but three gas consumers are unbounded or uncapped.	Broke via governor spam (High Z-03, Fixed) and remains Residual for gas (Medium Z-07).
GOV-1	Governance mistakes are recoverable.	The emergency admin’s powers are timelocked, announced and guardian-vetoable.	Broke: ownership was burned (High Z-06) and <code>setClaimGate</code> was neither delayed nor vetoable (Medium Z-08); both Fixed.

7 Findings Register

Summary

ID	Severity	Title	Confidence	Status
Z-01	High	Exact-output sells pay zero hook fee and mint free credit	PoC	Fixed
Z-02	High	Liquidation mark freezes while the perp book is empty	PoC	Fixed
Z-03	High	Unbounded governor scan bricks <code>relaunch()</code> for cents	PoC	Fixed
Z-04	High	<code>rescueSeeder</code> strands the active book and bricks re-birth	PoC	Fixed
Z-05	High	Block-number death clock destroys live tokens on L2	PoC	Fixed
Z-06	High	Registry ownership burned into the presale contract	PoC	Fixed
Z-07	Medium	<code>relaunch()</code> gas is unbounded in practice	PoC	Fixed
Z-08	Medium	<code>setClaimGate</code> froze migration instantly and un-vetoably	PoC	Fixed
Z-09	Medium	Crystal credit accrued on retired pools	Likely	Fixed
Z-10	Medium	Depth-derived risk caps are flash-manipulable	PoC	Residual
Z-11	Medium	Queued PLV exits are senior to live shares	Confirmed	Residual
Z-12	Medium	Reserve solvency degrades silently at the legacy clamp	Confirmed	Fixed
Z-13	Medium	Anti-snipe window is block-denominated	PoC	Accepted
Z-14	Medium	<code>blockhash</code> seed expiry stalls the gacha queue on L2	PoC	Accepted
Z-15	Low	<code>MagicFrensPeg</code> is free-mintable and grindable	Confirmed	Accepted
Z-16	Low	Presale refunds pay the original payer, not the holder	Confirmed	Residual
Z-17	Low	<code>legacyBuyStep</code> settles its intended, not actual, amount	Confirmed	Fixed
Z-18	Low	Un-added remainders strand in the registry	Confirmed	Residual
Z-19	Low	<code>liquidityForTokenOut</code> overflows on a degenerate band	PoC	Accepted
Z-20	Info	Gas reserves are unaudited magic numbers	Suspected	Accepted
Z-21	Info	<code>tx.origin</code> attribution under aliasing	Confirmed	Accepted
Z-22	Low	Reserve backing drifts a few tens of wei below its accounting	PoC	Residual
<i>Third pass — new vector + lead closure (§9):</i>				
Y-01	Medium	Reserve-ceiling breach voids the exit guarantee above $\sim 69\times$	PoC	Config
Y-02	Medium	Depth caps flash-manipulable (mechanism); PLV drain refuted	PoC	Residual

7.1 Z-01 — Exact-output sells pay zero hook fee and still mint crystal credit

Severity	High
Confidence	Confirmed with PoC
Status	Fixed
Location	<code>CauldronHook._afterSwap / _beforeSwap</code>
Proof	<code>test/attacks/Z01_ExactOutSellFeeBypass.t.sol</code>

Description. The hook takes its fee in ETH only. `_beforeSwap` charges exact-input buys; `_afterSwap` charges whenever ETH is the *unspecified* currency:

```
bool exactInput      = params.amountSpecified < 0;
bool unspecifiedIsEth = exactInput ? (!params.zeroForOne) : (params.zeroForOne);
if (!unspecifiedIsEth) return (BaseHook.afterSwap.selector, 0);
```

Enumerating the four quadrants:

Quadrant	amountSpecified	zeroForOne	Charged?
exact-in buy	< 0	true	yes, in <code>_beforeSwap</code>
exact-in sell	< 0	false	yes, in <code>_afterSwap</code>
exact-out buy	> 0	true	yes, in <code>_afterSwap</code>
exact-out sell	> 0	false	no — both skip it

`_beforeSwap` skips it for not being exact-input, and `_afterSwap` returns early because the unspecified leg is the *token*. The swap executes completely fee-free. Worse, the credit accrual in `_afterSwap` runs *before* that early return, so the swapper still banks crystal credit and lifetime volume.

Exploit path. Sell by naming the ETH output instead of the token input — a one-word change, and a standard aggregator route. Measured on the fork: an exact-input sell paid 4.24×10^{15} wei; the identical exact-output sell paid 0.

Impact. A total bypass of the protocol’s only revenue mechanism on the sell side. Every downstream sink is starved: the genesis dividend, the collection floor buy-back, the relaunch reserve (which *is* the next generation’s liquidity) and the proposer slice. Combined with the free credit accrual, a buy/sell round trip halves the intended cost of minting the NFT collection.

Fix (applied). The quadrant is refused in `_beforeSwap`. This is the correct resolution rather than a workaround: v4’s `afterSwapReturnDelta` applies to the *unspecified* currency, so an ETH fee physically cannot be taken on that leg, and charging in token would contradict the hook’s ETH-only fee design. Exact-input sells are economically equivalent and unaffected.

```
| if (!exactInput && !params.zeroForOne) revert ExactOutSellUnsupported();
```

7.2 Z-02 — The liquidation mark freezes whenever the perpetuals book is empty

Severity	High
Confidence	Confirmed with PoC
Status	Fixed
Location	PerpEngine._doSweep, _writeObs, twapTick
Proof	test/attacks/Z02_PerpStaleMark.t.sol

Description. The engine’s TWAP oracle is written only by `_pokeFunding`. On the keeperless path — the hook calling `sweepLiquidations` from `afterSwap` — that call sat *after* an early return:

```
if (_liqReentry) return;
uint256 len = _openIds.length;
if (len == 0) return; // <-- returns BEFORE _pokeFunding()
_pokeFunding(); // the only keeperless oracle write
```

So while nothing is open, no swap ever samples the oracle. `lastTick` stays frozen at the value it held when the book last emptied. `_writeObs` then integrates the entire quiet period at that stale tick (`tickCumulative += lastTick * dt`), and `twapTick` extrapolates the same stale tick across the whole lookback. The mark returns the pre-quiet-period price no matter how far spot has moved.

This is the same defect class the contract documents as fixed for the *intra-transaction* case; the inter-period case was still open because the oracle is simply not sampled.

Exploit path / measurement. With an empty book, an 8 ETH buy moved spot from $\sqrt{P} = 4.4169 \times 10^{32}$ to 3.1822×10^{32} (the token roughly doubling). One hour later the mark still read 4.4168×10^{32} — unchanged. A fully collateralised $2 \times$ long opened at that moment was liquidated *inside its own opening transaction* by its own post-open sweep: `positions[id].trader == address(0)` and `openCount == 0` immediately after a successful `openLong`.

Impact. Two directions. (i) The opener pays the 6.9% liquidation penalty on a position that was solvent at spot — a direct, repeatable loss to honest users, and a keeper reward plus Liquidator badge to whoever is watching. (ii) The mirror case: when the stale mark is favourable, a genuinely insolvent position is *unliquidatable* until the mark converges, and the shortfall is charged to the ETH PLV.

Fix (applied). `_pokeFunding()` is moved above the empty-book return, so every served swap samples the oracle whether or not anything is open. This is what makes the design genuinely keeperless; it costs one observation write on an otherwise idle sweep. The regression suite asserts the mark now tracks spot to within 1% after the same one-hour quiet period, and that the position survives.

7.3 Z-03 — Unbounded governor scan lets one NFT holder brick the rebirth

Severity	High
Confidence	Confirmed with PoC
Status	Fixed
Location	CauldronGovernor._recomputeLeader, _bestUnconsumed
Proof	test/attacks/Z03_GovernorSpamRelaunchDoS.t.sol

Description. `propose()` is permissionless for anyone holding one unit of voting power, with no cooldown, deposit or per-address cap, and `_recomputeLeader` scanned *every proposal ever submitted*. `relaunch()` reaches that scan up to three times in a single call:

CauldronRegistry.relaunch:597	governor.hasProposals()	-> _bestUnconsumed -> _recomputeLeader
CauldronRegistry.relaunch:660	governor.winner()	-> _bestUnconsumed -> _recomputeLeader
CauldronRegistry.relaunch:674	governor.markConsumed()	-> _recomputeLeader

The `markConsumed` recompute is unconditional on this path, because `relaunch` always consumes exactly the `id winner()` returned — i.e. the cached leader.

Exploit path / measurement. The scan cost grows strictly linearly: 5,239 gas at 1 proposal, 127,839 at 201, 495,639 at 801 — a marginal **613 gas per spam proposal, per scan**. Roughly 17,400 proposals exhaust a 32M budget across `relaunch`’s three scans, on top of the 6–10M gas of real rebirth work. At L2 prices that is a few hundredths of an ETH. The leader cache does not save it: a voter can keep `_leaderId` pointing at a still-open proposal, which forced `hasProposals()` from 1,464 gas to 196,766.

Impact. Permanent, permissionless halt of the eternal machine for a trivial cost. It compounds badly with Z-06: `registry.setGovernor` is `onlyOwner`, and ownership was burned into the presale, so a spammed governor could *never* be replaced. Recovery would have required a full V2 controller migration.

Fix (applied). `_recomputeLeader` now scans at most `MAX_LEADER_SCAN = 64` of the most recent proposals, making `relaunch` gas $O(1)$ in the proposal count. The cached-leader fast path is untouched, so an established winner keeps winning however much spam follows it, and a fresh mandate is by definition among the most recent proposals. Post-fix the scan is flat: identical gas at 201 and 801 proposals.

7.4 Z-04 — `rescueSeeder` permanently strands the active book and bricks rebirth

Severity	High
Confidence	Confirmed with PoC
Status	Fixed
Location	<code>CauldronSeeder.rescue</code> , <code>CauldronRegistry._removeLiquidity</code> , <code>migrateToSuccessor</code>
Proof	<code>test/attacks/Z04_SeederRescueStrandsLp.t.sol</code>

Description. The documented break-glass for an “aborted campaign” returned only the *loose* balances and then disarmed the campaign:

```
function rescue(address to) external onlyRegistry lock {
    ... transfer loose token + ETH ...
    seeding = false;                // <-- placed positions left in the pool
}
```

Every registry path to the only recovery function is gated on that same flag (if `(ISeeder(_seeder).seeding())` { `withdrawAll` }, in both `_removeLiquidity` and `migrateToSuccessor`), and `withdrawAll` is `onlyRegistry`. One call therefore made the placed liquidity unreachable forever.

The premise was false: `startSeed` places the two-sided base *and* the seed-floor slice in the same transaction, so a campaign holding only loose funds never exists. The hatch was always destructive.

Exploit path / measurement. On a progressive generation seeded with 10 ETH, the seeder held 1.534×10^{23} liquidity units across three tracked ranges. After `rescueSeeder()` that figure was *unchanged*, and `withdrawAll` reverted `OnlyRegistry` for every external caller. The escalation is worse than stranding: on a progressive generation the seeder’s book is the *only* source of `relaunch` ETH (`generationPositionId` is 0 and the reserve is single-sided token), so `relaunch()` then found `totalETH == 0` and reverted `NoLiquidityToSeed` — the machine could never be reborn at all.

A secondary defect compounds it: `startSeed` never cleared `ranges` (only `withdrawAll` did), so a post-rescue rebirth inherited the dead pool’s tick ranges (observed: 3 stale → 5 total instead of a clean reset).

Impact. Governance-triggerable, irreversible loss of an entire generation’s active liquidity, plus a permanent brick of the rebirth.

Fix (applied). `rescue` no longer clears `seeding`, so the registry’s teardown paths still reach `withdrawAll` and the book is recovered at the next `relaunch` or handoff; and `startSeed` now performs `delete ranges` as part of its per-campaign reset. The regression asserts that after a rescue the rebirth succeeds and every gen-1 range is fully unwound.

7.5 Z-05 — Block-denominated death clock destroys live tokens on the deployment target

Severity	High
Confidence	Confirmed with PoC
Status	Fixed
Location	<code>CauldronHook._getCurrentBucket</code> , <code>_recordVolume</code> , <code>getVolume24h</code>
Proof	<code>test/attacks/Z05_L2BlockClock.t.sol</code>

Description. The sliding volume window was counted in blocks, with constants pinned to Ethereum’s cadence:

```
uint256 public constant BLOCKS_PER_HOUR = 300; // "~12s blocks"
uint256 public constant BLOCKS_PER_DAY = 7_200; // 300 * 24
function _getCurrentBucket() private view returns (uint256) {
    return (block.number / BLOCKS_PER_HOUR) % HOURS_PER_DAY;
}
```

As §4 establishes, `block.number` on Nitro is the *parent chain’s* block number. On an Orbit L3 settling to Arbitrum One (≈ 0.25 s), 7,200 blocks is **30 minutes**, not 24 hours — the death clock runs $48\times$ too fast.

Exploit path. `relaunch()` is permissionless and its only other gate, `minLifetime`, is denominated in seconds. So: hold one MiFren, propose a brew, wait out the 3-day voting period, then kill the live token during its first 30-minute lull once it is an hour old. Demonstrated end to end by advancing `block.number` and `block.timestamp` independently: with 7,201 blocks elapsed but only 61 minutes of wall clock, a stranger’s `relaunch()` succeeded and retired a healthy pool.

Impact. Permissionless, permanent destruction of a live, liquid token by an unprivileged party — forced LP teardown, forced perp closure, forced migration of every holder. Both constants were `constant` (no setter, no migration path), and the hook had 58 bytes of EIP-170 headroom, so one could not have been added post-deployment. The pluggable `IDeathChecker` is only a partial mitigation, since the `volume24h` figure it receives is itself computed over the mis-calibrated window.

Fix (applied). The window is re-denominated in wall-clock seconds (`SECONDS_PER_HOUR / SECONDS_PER_DAY`), which carries the intended meaning identically on L1, on an Orbit L2 and on an Orbit L3. To reclaim the necessary bytes, the owner-only `trackPool` was removed — itself a small security improvement, since it let the owner hand-adopt a pool the registry never created, bypassing the `_afterInitialize` adoption gate. The hook now has 145 bytes of headroom. Regressions assert that 7,201 blocks no longer kill a healthy token, that a stranger’s `relaunch()` reverts `TokenStillAlive`, and that 24 hours of genuine silence still retires it.

7.6 Z-06 — Registry ownership is burned into the presale contract at ignition

Severity	High
Confidence	Confirmed with PoC
Status	Fixed
Location	deploy/DeployLaunchpad.s.sol:254, CauldronRegistry.summon
Proof	test/attacks/Z06_GovernanceLockout.t.sol

Description. The deploy script transferred registry ownership to the presale purely so that `finalize()` could reach `summon()`, which was `onlyOwner`:

```
| IOwnable(address(registry)).transferOwnership(address(presale));
```

But `MiFrensGenesis` calls exactly one registry function, exposes no generic forwarder and has no fallback. The transfer therefore did not delegate ownership — it *destroyed* it. From ignition onward no party, including the governance timelock, could ever call `setGovernor`, `setFactory`, `setSeeder`, `setSeedWindow`, `setReserveCeiling`, `setCollectionLedger`, `setNftMaxSupply`, `setRoyalty`, `setGenesisMetadata` or `setRedemptionExt`.

This directly contradicts the contracts’ own documentation, e.g. `setReserveCeiling`: “Owner = timelock; picked per iteration alongside the winning brew”; `setSeedWindow`: “Owner = timelock, chosen per iteration”. Neither is reachable even once.

Impact. Permanent loss of every economic tuning lever. Critically, both `governor` and `factory` are called from *inside* `relaunch()`, so a spammed governor (Z-03) or a reverting factory would brick the machine with no repair path short of a full V2 migration. Emergency-admin powers survive the handoff, so safety controls remain — which is why this is High rather than Critical.

Fix (applied). Ignition is now a distinct, narrow role. `CauldronBase` gains an appended `igniter` slot (appended, so registry/facet layout equality is preserved), `summon()` accepts the owner *or* the igniter, and the deploy script grants the presale the igniter role while transferring *ownership* to the governance timelock. The regression asserts the timelock can still replace the governor, retune the seed window and reserve ceiling, and that the igniter role confers no other power.

7.7 Z-07 — `relaunch()` gas is unbounded in practice

Severity	Medium
Confidence	Likely — no PoC
Status	Residual
Location	CauldronRegistry.relaunch, _perpHousekeep, CauldronHook.resolveTickets
Proof	—

Description. Two consumers inside `relaunch()` are neither bounded by a constant nor gas-capped:

1. `hook.forceClosePerps()` $\rightarrow$ `forceCloseAllDead()`, which performs a real settlement swap per position, up to `MAX_OPEN_POSITIONS = 64`. At roughly 350k gas per settlement that is $\approx 22\text{M}$ gas.
2. `hook.resolveTickets(300)`, which mints up to 300 NFTs at $\approx 80\text{k}$ each.

Both are wrapped in `try/catch`, but neither is called with a gas cap, so an out-of-gas child consumes 63/64 of the remaining gas and the `catch` resumes with 1/64 — far too little to finish the rebirth. Unlike the hook’s other sub-calls (which correctly forward `gasleft() - RESERVE`), the failure is not contained.

Exploit path. Fill the perpetuals book with 64 dust positions before death (`minCollateral` is 0.003 ETH, so ≈ 0.2 ETH total), and/or queue a large crystal backlog. `relaunch()` then reverts on gas.

Impact. Temporary denial of the rebirth. It is recoverable without governance: `forceCloseDead` and `resolveTickets` are both permissionless, so a keeper can drain the queues one at a time and then `relaunch`. Severity is Medium because the attacker must pay to re-arm it each cycle and cannot open new positions once the token is dead.

Recommendation (not applied). Cap both calls, mirroring the pattern the hook already uses elsewhere:

```
| try hook.forceClosePerps{gas: gasleft() - RELAUNCH_TAIL_RESERVE}() {} catch {}
```

and lower the `resolveTickets` budget inside `relaunch`. This was left unapplied because it alters `relaunch`’s failure semantics under partial drains and warrants its own fork-tested change rather than being bundled with the security fixes.

7.8 Z-08 — `setClaimGate` froze all migration instantly, un-announced and un-vetoable

Severity	Medium
Confidence	Confirmed with PoC
Status	Fixed
Location	<code>CauldronRegistry.setClaimGate</code>
Proof	<code>test/attacks/Z06_GovernanceLockout.t.sol</code>

Description. Setting a non-zero claim gate closes the 1:1 migration path for every ordinary holder — `claimByBurn`, `claimByBurnUpTo` and `autoMigrateBatch` all revert `VestingEnforced`. The setter carried `onlyEmergency` but *not* `timelocked`:

```
| function setClaimGate(address gate) external onlyEmergency {
|     claimGate = gate; // no arm, no delay, no veto window
| }
```

Unlike `setRedemptionPaused` — whose effect is deliberately overridden the instant an emergency is armed, via `_redeemBlocked()` — nothing forced the migration exit back open. The guardian’s veto only cancels an *armed* action, and there was none to cancel. The protocol’s advertised “exit guarantee” therefore did not cover migration at all.

Impact. A single un-announced call traps every previous-generation holder indefinitely. No theft: the gate must hold real old-generation tokens to migrate them.

Fix (applied). An asymmetric timelock. Restricting migration consumes the armed emergency timelock — making it announced, delayed and guardian-vetoable — while restoring instant migration remains immediate, so the safe direction is never slowed:

```
| if (gate != address(0)) _consumeTimelock();
```

7.9 Z-09 — Crystal credit accrued on retired generations’ pools

Severity	Medium
Confidence	Likely
Status	Fixed
Location	<code>CauldronHook._served, _afterSwap</code>
Proof	<code>test/attacks/Z06_GovernanceLockout.t.sol</code>

Description. `_served` asks only whether a pool is *tracked*, and `trackedPools` is never cleared, so every retired generation’s pool remains fully served forever. The credit block was gated on `trackedPools` but never

compared against `_liveKey`, so a swap on any past generation minted crystal credit in the *current* epoch and inflated `lifetimeVolumeOf` and `cumulativeVolume`.

Exploit path. A retired pool is empty after teardown, but Uniswap v4 liquidity is permissionless: an attacker re-provisions the dead pool with their own token and ETH, then round-trips against their own book. They control both sides, so there is no price impact and no market risk — only the flat fee — yet they accrue credit at the same rate as an honest trader taking real exposure to the live token, and can mint out the newborn collection without ever touching it.

Impact. Decouples NFT issuance from genuine participation in the live token, and corrupts the volume oracles the frontend and indexer read.

Fix (applied). Accrual is gated on the live pool’s token. Fees on retired pools still accrue (they are legitimate revenue); only credit is restricted.

```
if (collection != address(0) && absVolume > 0
    && Currency.unwrap(key.currency1) == Currency.unwrap(_liveKey.currency1)) {
```

7.10 Z-10 — Depth-derived risk caps are flash-manipulable

Severity	Medium
Confidence	Likely — no PoC
Status	Residual
Location	PerpEngine.activeEthDepth, _checkNotional, maxLeverage, _tryLiquidate
Proof	—

Description. `activeEthDepth()` reads `poolManager.getLiquidity(id)` — the liquidity *at the current tick*, an instantaneous quantity anyone can move by minting or burning a concentrated position. Every risk limit derives from it: the per-position notional cap (`maxNotionalBps`), the per-side OI cap (`maxOiBps`), the leverage tier ladder, and the per-timestamp liquidation cap (`maxLiqBps`).

Exploit paths. Two directions, both atomic from a contract (mint LP, act, burn LP — each in its own `unlock`):

1. *Inflate* depth to open a position far larger than the real book can absorb, then withdraw the liquidity. The exit swap then slips badly; the trader loses their collateral and the remainder is charged to insurance and then to LP principal.
2. *Deflate* depth to a small non-zero value, shrinking $cap = activeEthDepth \times maxLiqBps / 10^4$ below the position’s notional, so `liquidate` reverts `LiqCapped` and `_tryLiquidate` silently skips — protecting an underwater position from liquidation for as long as the manipulation is held. Note the guard is written `if (cap > 0 && ...)`, so driving depth to exactly zero *disables* the check rather than blocking; the exploitable region is small-but-non-zero.

Impact. Bypass of the bounded-slippage assumption that underwrites “no single position can create bad debt”, and a temporary liquidation-evasion primitive. Not directly profitable in isolation — the manipulator bears the LP cost — which is why this is Medium rather than High.

Recommendation (not applied). Derive the caps from a time-averaged depth, or from the protocol’s own position sizes (which the registry knows exactly), rather than from instantaneous in-range liquidity; and treat `cap = 0` as “block” rather than “allow”.

7.11 Z-11 — Queued PLV exits are senior to live shares

Severity	Medium
Confidence	Confirmed by construction
Status	Residual
Location	PerpVault.withdrawEth, claimPendingEth, assetsEth
Proof	—

Description. A withdrawal that exceeds the free buffer is booked as a fixed claim in `pendingEth` and subtracted from the share base: `assetsEth() = engine.totalEth() - pendingEth`. The source documents this as “queued ETH ... stops bearing bad-debt risk”, which is precisely the problem: `plv` can subsequently *fall* via `_absorbPlvLoss`, and the queued claim does not. Live shares absorb the entire loss while queued claims are made whole first.

Impact. A first-mover advantage that rewards running: any LP who suspects bad debt should withdraw immediately to convert into a senior fixed claim. In a stress event this is a bank-run incentive, not merely an accounting quirk.

Recommendation (not applied). Either mark queued claims down pro-rata when `_absorbPlvLoss` fires, or keep them in the share base until actually paid. This is a deliberate economic design decision rather than an implementation bug, so it is reported for the protocol owners rather than unilaterally changed.

7.12 Z-12 — Reserve solvency degrades silently at the legacy clamp

Severity	Medium
Confidence	Likely — no PoC
Status	Residual
Location	CauldronRegistry.relaunch (steps 8, 8b)
Proof	—

Description. The reserve backs three claimants at once: 1:1 migration, the genesis floor, and the collection legacy floors. Sizing is:

```
uint256 newActive = tokensFromLP > unclaimedGenesis ? tokensFromLP - unclaimedGenesis : tokensFromLP;
if (newActive == 0) newActive = GEN1_ACTIVE_TOKENS; // (a)
uint256 legacy = collectionLedger.totalEntitled();
newActive = newActive > legacy ? newActive - legacy : newActive; // (b)
if (newActive > TOTAL_SUPPLY) newActive = TOTAL_SUPPLY;
uint256 newReserve = TOTAL_SUPPLY - newActive;
```

Branch (b) is the defect: when $legacy \geq newActive$ the subtraction is skipped *entirely* rather than clamped to zero, so the reserve is not enlarged for the legacy claim at all. Branch (a) is worse in the degenerate case: a zero recovery substitutes a fixed 80% constant that bears no relation to the circulating supply needing migration.

Impact. The three claimants compete first-come-first-served on an under-collateralised pot. Late migrators hit “reserve short” and are permanently unable to migrate 1:1. Reachable after enough generations of accumulated entitlement, or after an `emergencyWithdrawLP` has emptied a generation’s LP.

Recommendation (not applied). Make the shortfall explicit and observable: emit a diagnostic when the clamp binds, and prefer proportional haircutting across the three claimants over silent first-come-first-served. Changing the allocation policy is an economic decision for the protocol owners.

7.13 Z-13 — The anti-snipe window remains block-denominated

Severity	Medium
Confidence	Confirmed with PoC
Status	Accepted
Location	CauldronHook._defaultSurtaxBps, poolInitBlock, snipeWindowBlocks
Proof	test/attacks/Z05_L2BlockClock.t.sol

Description. The launch surtax decays over `snipeWindowBlocks` counted in `block.number`. By §4 the default of 30 is ≈ 6 minutes on an Orbit L2 but ≈ 7.5 seconds on an Orbit L3.

Impact. A materially weaker launch defence than the design intends. It is *not* in the same class as Z-05: the failure mode is a shorter surtax ramp, not the destruction of a live token, and `setSnipeParams` is owner-tunable, so it can be corrected on the live chain once the parent cadence is known.

Disposition. Accepted and documented as a **deployment-configuration requirement**: measure the parent-chain cadence on Robinhood and set `snipeWindowBlocks` to the block count matching the intended wall-clock window before the first summon. Note also that the jitter term reads `blockhash(block.number - 1)`, which is documented-insecure on Nitro; this is benign here because the jitter can only ever *raise* the rate and the decay term dominates.

7.14 Z-14 — blockhash seed expiry stalls the gacha queue on a fast parent chain

Severity	Medium
Confidence	Confirmed with PoC
Status	Accepted
Location	CauldronHook._resolveTickets, CauldronCollection.reveal, MiFrensGenesis.reveal
Proof	test/attacks/Z05_L2BlockClock.t.sol

Description. Ticket outcomes are seeded by `blockhash(commitBlock)`, valid for 256 blocks. On an Orbit L3 that horizon is ≈ 64 seconds, so expiry becomes routine rather than exceptional. The expiry path is:

```
| if (bh == 0) { b.commitBlock = uint48(block.number); break; } // re-anchor, FIFO
```

Demonstrated: after 257 blocks, `resolveTickets(10)` processed 0 tickets and silently re-anchored the batch.

Impact. Two effects. (i) The `break` head-of-line blocks the entire FIFO queue behind one expired batch. (ii) The re-anchor is an additional draw for that batch’s owner. The design deliberately re-anchors rather than substituting a deterministic fallback — which is the right call, since a deterministic fallback would be a *predictable* second draw — so the residual is a liveness and fairness wrinkle, not a grinding vulnerability. The pity counter further limits the value of a re-roll.

Disposition. Accepted; recommend a keeper that resolves promptly, and consider skipping past an expired batch rather than `breaking` so one stale entry cannot stall the queue.

7.15 Z-15 — MagicFrensPeg mints for free and its randomness is grindable

Severity	Low
Confidence	Confirmed
Status	Accepted
Location	MagicFrensPeg.buyFren, sellFren, _random
Proof	—

Description. Three defects in one legacy contract: `buyFren()` is payable but never checks `msg.value`, so tokens and NFTs mint for free; `sellFren()` refunds `msg.value` — the caller’s own ETH, on a non-payable path, hence always zero; and `_random` draws on `block.prevrandao`, a constant on Arbitrum, making trait rarity fully predictable and grindable by a contract that reverts on a bad roll.

Disposition. Accepted with a hard caveat. This file is excluded from the `cauldron` build profile and is not part of the Cauldron deployment. It must **not** be deployed as written. No fix applied, because fixing an unused contract would enlarge the audited surface without benefit.

7.16 Z-16 — Presale refunds pay the original payer regardless of NFT ownership

Severity	Low
Confidence	Confirmed
Status	Residual
Location	MiFrensGenesis.cancelPresale, refund
Proof	—

Description. `refund()` returns `paid[msg.sender]`, recorded at mint time. A minter who sells their MiFrens on the secondary market before a cancellation still collects the full refund, while the secondary buyer receives nothing and holds an NFT for a collection that will never launch. `cancelPresale` is deployer-only, untimelocked, and callable even when the presale is fully sold out but not yet ignited.

Recommendation (not applied). Escrow the refund against the NFT (burn-to-refund), or disable cancellation once `soldOut()`. Not applied because it changes presale economics, which is an owner decision.

7.17 Z-17 — legacyBuyStep settles its intended amount, not the actual delta

Severity	Low
Confidence	Confirmed
Status	Residual
Location	CauldronHook.legacyBuyStep
Proof	—

Description. The buy-back settles the constant `amt` rather than the swap’s realised debit:

```
BalanceDelta d = poolManager.swap(key, SwapParams({... amountSpecified: -int256(amt) ...}), "");
poolManager.settle{value: amt}(); // should be uint256(uint128(-d.amount0()))
```

If the price limit ever binds, the pool consumes less than `amt` and the hook over-settles, leaving a positive delta that is never taken — which, since deltas must net to zero at `unlock` close, would revert the *user’s* parent swap. Every other settlement site in the codebase (`PoolOps.executeBuy`, `PerpEngine._swapBody`) correctly uses the realised delta.

Impact. Unreachable in practice: `amt` is bounded by `legacyThreshold` (0.02 ETH by default) against a full-range book, so `MIN_SQRT_LIMIT` cannot bind. Reported as a robustness defect and a latent hazard if the threshold is ever raised or the book made much thinner.

Recommendation (not applied). Settle `uint256(uint128(-d.amount0()))` and return any remainder to `legacyBuffer`. Deferred only because the hook’s EIP-170 headroom was reserved for the Z-01 and Z-05 fixes.

7.18 Z-18 — Un-added remainders strand in the registry

Severity	Low
Confidence	Confirmed
Status	Residual
Location	<code>PoolOps.buyCollection</code> , <code>RedemptionExt._pullGrow</code>
Proof	—

Description. Both pull a payment from the caller and then call `addToReserve`, which returns the amount the position *actually* consumed. Accounting correctly credits only that amount — so the floor is never overstated — but the difference (and the whole payment, when liquidity rounds to zero) simply remains in the registry with no accounting entry and no path out except `emergencySweep`.

Recommendation (not applied). Refund *paid* – *added* to the payer, or require *added* > 0.

7.19 Z-19 — `liquidityForTokenOut` overflows on a degenerate band

Severity	Low
Confidence	Confirmed with PoC
Status	Accepted
Location	<code>ReserveLib.liquidityForTokenOut</code>
Proof	<code>test/invariants/ZPureInvariants.t.sol</code>

Description. The function casts to `uint128` and reverts `SafeCastOverflow` once $amount \times Q_{96} / (\sqrt{P_{hi}} - \sqrt{P_{lo}}) > 2^{128}$, i.e. for a narrow band at the bottom of tick space. A revert on this path inside `relaunch()` would freeze the machine, exactly like the zero-liquidity case the code already guards.

Disposition. Accepted. Fuzzing across the protocol’s entire launch-price domain confirms the cast always has headroom, because `reserveTicks` always spans from `MIN_TICK` and $(\sqrt{P_{hi}} - \sqrt{P_{lo}})$ is consequently enormous. Reaching the overflow would require a launch price below roughly 10 wei of ETH against the full 777M supply. Both the safe domain and the degenerate boundary are pinned by tests.

7.20 Z-20 — In-swap gas reserves are unaudited magic numbers

Severity	Info
Confidence	Suspected
Status	Accepted
Location	<code>LIQ_GAS_RESERVE</code> , <code>GACHA_GAS_RESERVE</code> , <code>LEGACY_GAS_RESERVE</code> , <code>SEED_POKE_GAS_RESERVE</code>
Proof	—

Description. Four hand-tuned constants (180k–350k) decide how much gas `afterSwap` keeps for itself before forwarding the rest to a side effect. The pattern is sound — forwarding `gasleft()` – `R` means an out-of-gas child consumes exactly its budget — but the reserves were calibrated empirically on an L1-like fork, and the nesting can be deep: `afterSwap` → `sweepLiquidations` → nested swap → `afterSwap` → `legacyBuyStep` → nested swap.

Disposition. Accepted; recommend re-measuring worst-case tail cost on the target chain and asserting the margin in a test, since the failure mode is a reverted user swap.

7.21 Z-21 — `tx.origin` attribution under L2 aliasing

Severity	Info
Confidence	Confirmed
Status	Accepted
Location	<code>CauldronHook._afterSwap</code> , <code>sweepLiquidations(tx.origin)</code>
Proof	—

Description. Untagged swaps credit crystals to `tx.origin`, and the in-swap liquidator reward is paid to `tx.origin`. The source’s rationale is sound for the dominant case (a router would otherwise receive the user’s NFT). On Arbitrum, an L1→L2 retryable executes with the *aliased* L1 sender as origin, so a keeper

reward can be credited to an address reachable only from L1. `_payOut` sends with a 30k budget and credits `payoutOwed` on failure, so nothing is destroyed — but a plain send to a code-less alias succeeds, leaving the ETH only L1-recoverable.

Disposition. Accepted; `creditUntaggedSwaps` can disable the crystal half.

7.22 Z-22 — Reserve backing drifts a few tens of wei below its accounting

Severity	Low
Confidence	Confirmed with PoC (rare fuzz counterexample)
Status	Residual
Location	<code>PoolOps.addToReserve</code> , <code>claimFromReserve</code> , <code>ReserveLib</code>
Proof	pre-existing <code>invariant_reserveBacksItsClaims</code>

Description. Found by the repository’s own stateful invariant suite during the post-remediation verification run, and reproduced here for completeness:

```
I-3: the reserve holds less than it owes (drained backing):
390518549645138909242788556 < 390518549645138909242788645
```

A shortfall of **89 wei** against a reserve of 3.9×10^{26} — a relative error of 2×10^{-25} . The mechanism is the liquidity rounding already characterised in INV RES-2/RES-3: reserve capacity is measured through `ReserveLib.tokenOutForLiquidity`, which rounds *down*, so the measured backing sits up to one liquidity unit’s worth of token (order 10^3 wei at realistic launch prices) below the position’s true content.

Impact. Immaterial in practice, and *not* a solvency defect: every claim path carries a `CLAIM_DUST` tolerance of 10^{12} wei — ten orders of magnitude larger than the observed drift — so no redeemer or migrator is ever short-changed, and `migrateOne` / `redeemOgFren` still revert rather than under-deliver. The practical consequence is a test-suite one: the invariant asserts the relation with *zero* tolerance, so it is intermittently red under fuzzing (observed once in roughly nine runs; four fresh seeds afterwards all passed), and Foundry then replays the cached counterexample on every subsequent run until `cache/invariant` is cleared.

Attribution. This is a pre-existing property, not a regression from the remediation: none of the eight applied changes touches `PoolOps`, `ReserveLib`, `RedemptionExt` or any reserve accounting path. The fixes do alter which call sequences the fuzzer reaches (Z-01 rejects a swap quadrant; Z-05 changes when a pool reads dead), which is the likely reason the counterexample surfaced in this engagement rather than an earlier one.

Recommendation (not applied). Give the invariant the same dust tolerance the protocol itself uses — the sibling `invariant_migrationShortfallIsDustOnly` already does exactly this — rather than changing any production code. We deliberately did not edit the pre-existing assertion: weakening someone else’s invariant to green a run is the wrong instinct, and the drift deserves to be recorded rather than silenced.

8 Failed Attacks and Safety Arguments

Attacks we attempted and could *not* make work. Each is recorded with the property that defeats it, because these are the load-bearing guarantees.

SA-1 — Shrinking the next generation’s reserve by donating to the dying pool. `PoolOps.removeAll` collects accrued fees along with principal, so `poolManager.donate()` inflates `tokensFromLP` and hence `newActive`, shrinking `newReserve` one-for-one. This looked like a cheap way to strand other holders’ migration. It is not, because the donation is conserved. Let C be circulating supply, L the amount in the LP, g the unclaimed genesis, with $C + L = TOTAL$. Then $newReserve = TOTAL - (L - g) = C + g$: exactly the circulating supply needing migration, plus genesis. Donating X moves X from C into L , giving $C' = C - X$ and $newReserve' = C - X + g$ — still exact. The attacker destroys their own tokens and strands nobody. The `newActive > TOTAL_SUPPLY` clamp is therefore defensive only; it cannot be reached without minting.

SA-2 — Self-liquidation for profit. A liquidator receives `keeperBps = 145 bps of the penalty`, and the penalty is 6.9% of collateral — so roughly 0.1% of collateral is returned against a 6.9% charge. Liquidating one’s own position is strictly loss-making. The Liquidator badge is the real incentive, and it is not fungible.

SA-3 — Guardian permanently disabling the break-glass. A hostile guardian can veto every armed action, apparently freezing emergency recovery forever. It cannot: `setGuardian` is reachable by the `emergencyAdmin` and is *not* itself timelocked, so the timelock replaces the guardian and proceeds. The veto is genuinely pure-upside.

SA-4 — First-depositor inflation of the PerpVault. Shares are minted with a 10^6 virtual offset, and — decisively — there is no permissionless path to inflate `plv`: `fundPlv` is `onlyOwner`, `fundFromVault` is `onlyVault`, and `creditPerpFee` is hook-only. The only attacker-reachable inflation is routing perp fees, which costs more than it moves.

SA-5 — Forging the facet’s return data or hijacking the delegatecall. `setRedemptionExt` is one-shot, rejects code-less targets (a `delegatecall` to an empty account returns success with no data, which would make every redemption a silent no-op), and the forwarder bubbles return data verbatim. The forwarders are deliberately *not* `nonReentrant` because the facet’s own guard runs against the registry’s shared slot; adding one would double-lock. We confirmed the layouts are identical by construction and that the remediation preserves this (INV-F).

SA-6 — Vesting-grant spam to DoS a holder’s `claim()`. `vestBatch` pulls `min(balance, allowance)` in one go, so repeating a victim’s address within a batch yields zero on every subsequent iteration. Grant count is bounded by the victim’s own deposit actions.

SA-7 — Double-voting across the L2 checkpoint granularity. Because `block.number` is coarse on Nitro, many transfers collapse into one OpenZeppelin checkpoint, so we tested whether votes could be duplicated by shuffling NFTs inside a parent-block window. They cannot: `vote` requires `block.number > p.snapshot` and reads `getPastVotes` at that fixed snapshot, so post-snapshot movement cannot mint voting power. The coarse clock only delays the opening of voting by one parent block.

SA-8 — Claiming an exempt tier by forging `hookData`. A direct swapper controls `hookData` freely, so we attempted to tag a tax-exempt address. `_isExemptPlayer` requires `isOpenener[sender]` and `taxExempt[player]`, and `_taxedPlayer` ignores the tag unless the sender is a trusted opener. The exemption cannot be claimed by an untrusted caller.

SA-9 — Sandwiching the hook’s own buy-back. The trigger (`legacyBuffer >= legacyThreshold`) is publicly computable, and an attacker can trip it with their own buy so the hook buys immediately afterwards at the inflated price. This works, but the extractable value is bounded by the buffer (0.02 ETH by default), and the mempool-less sequencer removes the classical variant. Recorded under Z-17 rather than as a finding in its own right.

SA-10 — Reaching the $69\times$ reserve through the seeder. `CauldronSeeder` custodies only what `startSeed` handed it and holds no reference to the reserve position, which is a registry-owned `PositionManager` NFT. The separation is structural, not procedural (INV SEED-1).

9 Third Pass — Final Red-Team (Lead Closure + New Vector)

A third, adversarial pass took every lead the two prior passes left open and drove each to a definitive verdict, then hunted the next vector in the system seams. It introduced the two harness primitives the earlier work lacked — a raw `modifyLiquidity` (mint/burn a concentrated band) and directional pumps that push spot *through* the $69\times$ reserve ceiling — which is precisely why the remaining leads had been unreproducible. Tests live in `test/attacks/Y0*` (harness `YBase.sol`); the standalone writeup is `test/attacks/EXPLOIT_REPORT_PASS3.md`.

9.1 Lead ledger

ID	Title	Prior	Pass-3 verdict
Y-01	Reserve-ceiling breach voids the exit guarantee above $\sim 69\times$	— (new)	Confirmed Medium
Y-02a (Z-10)	Depth-derived risk caps are flash-manipulable	Likely/ Residual	Confirmed (mechanism)
Y-02b (Z-10)	... can it drain the PLV?	Likely/ Residual	Refuted (token scarcity)
Z-07	<code>relaunch()</code> gas unbounded $\rightarrow$ brick	Likely/ Residual	Confirmed $\rightarrow$ Fixed
Z-12	Reserve solvency degrades silently at the legacy clamp	Likely/ Residual	Confirmed $\rightarrow$ Fixed
Z-17	<code>legacyBuyStep</code> settles intended, not realised, delta	Confirmed/ Residual	Fixed
Z-11	Queued PLV exits senior to live shares	Confirmed/ Residual	Carried (owner policy)
Z-13/14/21	Block-denom. anti-snipe / gacha expiry / <code>tx.origin</code> alias	Accepted	Carried (L2 config)
Z-16/18/20/22	Refund payee / stranded remainders / gas consts / 89-wei drift	Residual	Carried (immaterial)

9.2 Y-01 — Reserve-ceiling breach voids the exit guarantee above $\sim 69\times$

Severity	Medium
Confidence	Confirmed with PoC
Status	Config + partial fix
Location	ReserveLib.reserveTicks, PoolOps.claimFromReserve/addToReserve, RedemptionExt.redeemOgFren, CauldronRegistry.claimByBurn
Proof	test/attacks/Y01_ReserveCeilingBreach.t.sol

Description. The migration/genesis/legacy reserve is a single *out-of-range*, single-sided token1 position placed *below* the launch tick (`tickUpper = alignDown(launchTick - offset)`, default `offset = 42400  $\approx 69\times$`). Every reserve-side operation assumes spot stays *above* `tickUpper` so the band is pure token:

```
// claimFromReserve: "reserve is out of range so ETH out = 0" (amount1Min = 0)
uint128 liquidity = ReserveLib.liquidityForTokenOut(tickLower, tickUpper, amount);
// addToReserve: single-sided top-up, max0 (ETH) = 0
params[0] = abi.encode(positionId, liquidity, uint128(0), uint128(amount), "");
```

`liquidityForTokenOut` sizes L across the *whole* band. Once the token appreciates past the ceiling and spot trades *into* the band, removing that L delivers only $amount \cdot (\sqrt{P} - \sqrt{Lo}) / (\sqrt{Hi} - \sqrt{Lo})$ of token and returns the remainder as *ETH*, so every claim short-delivers and trips its own H-03 guard (`migrateOne: if (got + CLAIM_DUST < amount) revert; redeemOgFren: if (amount + 1e12 < F) revert NoBalance`), and `addToReserve` reverts outright because it declares no ETH leg.

PoC (all green on current code). Launch tick 172,529, ceiling `tickUpper = 130,000`; a 200 ETH buy ($10\times$ the 20 ETH seed) drives spot to tick 128,279. Then: (i) `redeemOgFren` reverts for every remaining OG while `floorPerFren()` still advertises the full 1.5385×10^{24} ; (ii) `donateToReserve` (the floor-ratchet path) reverts; (iii) a gen-1 holder’s `claimByBurn` reverts “reserve short”. A one-spacing breach cannot leak the ETH leg (shortfall $\sim 1.5 \times 10^{22}$ wei, eleven orders above `CLAIM_DUST`); and the next `relaunch` self-heals by re-parking a fresh out-of-range reserve.

Impact. The advertised exit guarantee (1:1 migration + genesis floor) is silently voided *while* the token trades above its $69\times$ ceiling, exactly when holders most want to exit, and the floor view keeps promising the full number. Reachable by a plain buy (value not burned) or a genuine bull run. Bounded: it self-heals at `relaunch` and cannot leak ETH.

Fix. (1) *Config*: raise `nextReserveCeilingOffset` per iteration via the existing `setReserveCeiling` (bounded [4000, 138000]) so the ceiling sits above any plausible appreciation — a deeper ceiling costs only conversion optics, not solvency. (2) *Observability (APPLIED)*: `CauldronRegistry.floorClaimableNow()` returns (`bool claimable`, `uint256 perFren`) by comparing spot tick to the band’s `tickUpper`, so the UI shows “floor temporarily out of range” rather than a bare revert; its signal matches `redeemOgFren`’s reachability exactly (cost 299 B; registry 24,097 B, 479 free). (3) *Structural (owner call)*: let the claim paths accept the ETH leg of an in-range removal and credit it to the floor, degrading gracefully instead of reverting.

9.3 Y-02 — Depth-derived risk caps are flash-manipulable (resolves Z-10)

Severity	Medium
Confidence	Confirmed (mechanism); Refuted (drain)
Status	Bounded
Location	PerpEngine.activeEthDepth, _checkNotional, openLong, _tryLiquidate
Proof	test/attacks/Y02_DepthManip.t.sol

Confirmed (Y-02a). `activeEthDepth()` reads `poolManager.getLiquidity(id)` (instantaneous in-range L) and every risk cap derives from it. Minting an *asymmetric* in-range band (`tickLower` one spacing below spot $\rightarrow$ tiny token leg; `tickUpper` far above $\rightarrow$ large ETH leg recovered on burn) lifted measured depth $20 \rightarrow 91.7$ ETH; the same open that correctly reverted `BadLeverage` at the true depth then succeeded, and after burning the band the live position’s notional (1.86×10^{18}) exceeds the per-position cap at the real depth (1.03×10^{18}).

Refuted as a drain (Y-02b) — stronger than “not profitable.” The perp token is single-sourced from the very pool whose depth is measured, so inflating depth by factor k costs token $\propto k$: the $4.6\times$ inflation above consumed 3.64×10^{25} tokens ($\approx 4.7\%$ of the 7.77×10^{26} fixed supply) for a *single* band. Cratering the true book to force bad debt needs an order of magnitude more — an unreachable fraction of supply — so with a realistic budget the PLV+insurance pot does not fall ($60.00 \rightarrow 60.09$ ETH). Direction-2 (“deflate depth to evade liquidation”) is structurally refuted for the green-candle book: the protocol’s full-range active LP is a hard floor on in-range depth that an attacker cannot remove.

Residual. Time-averaged depth (the Z-10 recommendation) remains the right hardening *if* a future iteration seeds a thin, concentrated active book (e.g. a progressive launch), where the scarcity bound weakens.

9.4 Fixes applied in this pass

ID	File	Change
Z-07	CauldronRegistry	Cap <code>forceClosePerps</code> and <code>resolveTickets</code> inside <code>relaunch()</code> to <code>gasleft() - RELAUNCH_TAIL_RESERVE</code> (8M); survivors cleared by the permissionless paths
Z-12	CauldronRegistry	Legacy clamp is now subtract-or-zero with a <code>ReserveShortfall</code> diagnostic event, not a silent skip; active-pool floor preserved
Z-17	CauldronHook	<code>legacyBuyStep</code> settles the <i>realised</i> ETH debit and refunds the unspent remainder to <code>legacyBuffer</code>

Regressions. `Y03_RelaunchGasBrick` drives a maximally-spammed 64-position book through `relaunch()` under a hard 24M-gas cap (below the ~29M a full force-close needs) and asserts the rebirth completes and re-arms — the pre-fix brick condition. The full attack suite (37 pre-existing + 9 new Y-tests), all invariant suites, `LifecycleE2E` (rebirth to round 3 + legacy buyback + progressive perps) and the launchpad suite stay green. Post-fix sizes: hook 24,477 B (99 free), registry 23,798 B (778 free) — both under EIP-170.

10 Remediation Summary

ID	File	Change
Z-01	CauldronHook	Reject exact-output sells in <code>_beforeSwap</code> (<code>ExactOutSellUnsupported</code>)
Z-02	PerpEngine	Move <code>_pokeFunding()</code> above the empty-book early return in <code>_doSweep</code>
Z-03	CauldronGovernor	Bound <code>_recomputeLeader</code> to <code>MAX_LEADER_SCAN = 64</code>
Z-04	CauldronSeeder	<code>rescue</code> no longer clears seeding; <code>startSeed</code> does delete ranges
Z-05	CauldronHook	Volume window re-denominated in seconds; <code>trackPool</code> removed to fund the bytes
Z-06	CauldronBase, CauldronRegistry, DeployLaunchpad	New <code>igniter</code> role; <code>summon</code> accepts owner or igniter; ownership goes to the timelock
Z-08	CauldronRegistry	<code>setClaimGate</code> consumes the timelock when restricting; instant when restoring
Z-09	CauldronHook	Credit accrual gated on the live pool’s token
<i>Applied in the third pass (§9):</i>		
Z-07	CauldronRegistry	Gas-cap <code>forceClosePerps</code> + <code>resolveTickets</code> in <code>relaunch()</code>
Z-12	CauldronRegistry	Legacy clamp → subtract-or-zero + <code>ReserveShortfall</code> event
Z-17	CauldronHook	<code>legacyBuyStep</code> settles the realised delta; refunds the remainder

10.1 EIP-170 impact

`CauldronHook` was 58 bytes under the limit before this work and briefly exceeded it by 41 bytes; removing `trackPool` and the two now-meaningless block constants brought it to 145 bytes of headroom. The third-pass fixes (Z-07/Z-12/Z-17 plus the Y-01 `floorClaimableNow()` view) consumed some of that: **final runtime sizes are hook 24,477 (99 free) and registry 24,097 (479 free)**, with engine 24,510 (66 free); governor 6,389; seeder 10,665; facet 6,523. The engine and hook remain close enough to the limit that any further change should be budgeted before it is designed.

10.2 Storage-layout safety

The only new state variable, `igniter`, was **appended to `CauldronBase`** rather than added to `CauldronRegistry`. Both the registry and `RedemptionExt` therefore continue to declare no state of their own, and the delegatecall pair’s layouts remain identical by construction (INV-F).

11 Recommendations Before Mainnet

1. **Deferred fixes — now applied (§9):** Z-07 (both uncapped relaunch sub-calls are gas-capped), Z-12 (reserve shortfall is clamped-to-zero and emitted as `ReserveShortfall`) and Z-17 (`legacyBuyStep` settles the realised delta). Z-10 remains as a residual: apply time-averaged depth for the risk caps *if* a future iteration seeds a thin, concentrated active book, where token scarcity no longer bounds the manipulation (§9, Y-02).
2. **Reserve ceiling (Y-01):** set `nextReserveCeilingOffset` per iteration via `setReserveCeiling` so the ceiling sits above any plausible appreciation — the default $69\times$ silently voids the exit guarantee while the token trades above it. The `floorClaimableNow()` view (applied this pass) lets the frontend distinguish “out of range” from “insolvent”; wire it into the redeem UI (§9).
3. **Measure the parent-chain cadence on Robinhood** and set `snipeWindowBlocks` accordingly before the first summon (Z-13). Re-measure the in-swap gas reserves on the target chain (Z-20).
4. **Validate blockhash behaviour on the target chain** before relying on the gacha for value. If seeds prove predictable within the commit window, move to a VRF or a two-phase commit with an external beacon (§4).
5. **Set a non-zero EMERGENCY_DELAY.** The deploy script defaults to 48 hours, but the harness path constructs with 0, which makes every “timelocked” action instant. The delay is immutable once constructed.
6. **Do not deploy MagicFrensPeg (Z-15).**
7. **Re-run this suite against a Nitro fork** once one is available; the Sepolia fork cannot exercise the L2 semantics that drive Z-05, Z-13, Z-14 and Z-21.

A Test Suite and Results

A.1 Commands

```
export FORK_RPC=https://ethereum-sepolia-rpc.publicnode.com \
    POOL_MANAGER=0xE03A1074c86CFedD5C142C4F04F1a1536e203543 \
    POSITION_MANAGER=0x429ba70129df741B2Ca2a85BC3A2a3328e5c09b4

FOUNDRY_PROFILE=cauldron forge test --match-path 'test/attacks/Z0*' -vv
FOUNDRY_PROFILE=cauldron forge test --match-path 'test/invariants/Z*' -vv
FOUNDRY_PROFILE=cauldron forge test # full suite
```

A.2 Result

302 tests passed, 0 failed, 0 skipped, across 57 suites (86.3s wall, 604s CPU). The 39 tests added by this engagement are listed below; the remainder are the repository’s pre-existing suites, which also pass after the remediation.

One caveat, stated plainly: on one of several verification runs the repository’s pre-existing `invariant_reserveBacksItsClaim` produced an 89-wei counterexample. It is analysed as finding Z-22 — a zero-tolerance assertion over a rounding-bounded quantity, not a solvency defect — and it passed on four subsequent fresh seeds. Note that Foundry persists invariant counterexamples under `cache/invariant` and *replays* them on every later run regardless of `-fuzz-seed`, so that directory must be cleared before a re-run is meaningful.

Suite	Test	Result
Z01_ExactOutSellFeeBypass	<code>test_FIXED_ExactOutputSell_IsRejected</code>	pass
	<code>test_FIXED_ExactOutputSell_MintsNoFreeCredit</code>	pass
	<code>test_SAFE_OtherThreeQuadrantsAreCharged</code>	pass
Z02_PerpStaleMark	<code>test_FIXED_MarkTracksSpotWithAnEmptyBook</code>	pass
	<code>test_FIXED_PositionSurvivesOpenAfterQuietPeriod</code>	pass
	<code>test_SAFE_WithKeeperPoke_PositionSurvives</code>	pass
Z03_GovernorSpamRelaunchDoS	<code>test_FIXED_RelaunchScanIsBoundedUnderSpam</code>	pass
	<code>test_FIXED_ForcedRescanStaysBounded</code>	pass
	<code>test_INFO_ProposingIsStillPermissionless</code>	pass
Z04_SeederRescueStrandsLp	<code>test_FIXED_RescueLeavesTheRecoveryPathOpen</code>	pass
	<code>test_FIXED_RelaunchStillRecoversAfterRescue</code>	pass

Suite	Test	Result
Z05_L2BlockClock	test_FIXED_RangesResetPerCampaign	pass
	test_SAFE_NormalRelaunchRecoversTheBook	pass
	test_FIXED_L2_BlockAdvanceDoesNotKillAHealthyToken	pass
	test_FIXED_L2_HealthyTokenIsNotRelaunchable	pass
	test_FIXED_L2_RealDayOfSilenceStillKills	pass
	test_RESIDUAL_L2_AntiSnipeWindowIsStillBlockBased	pass
	test_RESIDUAL_L2_TicketSeedExpiryReAnchors...	pass
Z06_GovernanceLockout	test_SAFE_TicketResolvesInsideTheSeedWindow	pass
	test_FIXED_IgniterRoleKeepsOwnershipWithGovernance	pass
	test_FIXED_ClaimGateRequiresArmedTimelock	pass
	test_FIXED_RetiredPoolEarnsNoCrystalCredit	pass
	test_SAFE_EmergencyAdminPowersSurviveTheHandoff	pass
ZPureInvariants (fuzz)	testFuzz_INV_R_ReserveNeverOverDelivers	pass (257)
	testFuzz_INV_D_RoundTripDustIsBounded	pass (257)
	testFuzz_INV_R_ReserveTicksAlwaysValid	pass (256)
	testFuzz_INV_R_LiquidityCastIsSafeInProtocolDomain	pass (256)
	test_INV_R_LiquidityCastOverflowsOnDegenerateBasis	pass
	testFuzz_INV_S_ScheduleMonotoneBoundedAndTerminates	pass (256)
	testFuzz_INV_S_PokeFrequencyCannotAccelerate	pass (256)
ZLedgerInvariants (stateful)	invariant_L_TotalEntitledEqualsSumOfGenerations	pass
	invariant_L_PayoutsNeverExceedCredits	pass
	invariant_L_FloorNeverExceedsPot	pass
ZSystemInvariants (stateful, fork)	invariant_SUP1_TokenIsNonMintable	pass
	invariant_HOOK1_EthBooksAreSolvent	pass
	invariant_PERP1_EngineEthBooksAreSolvent	pass
	invariant_PERP2_TotalEthMatchesDefinition	pass
	invariant_PERP3_TokenInventoryIsAccounted	pass
	invariant_RES1_GenesisFloorIsBacked	pass

The stateful ledger run executed **128,000 handler calls with zero reverts**. The fork-based system run drove buys, exact-input sells, exact-output sells, perpetual opens and closes, liquidations, oracle pokes and ticket resolution in random order; because v4-core reverts **unlock** on any unsettled delta, every successful handler call is also a proof that its V4 accounting settled (INV V4-1).

B Harness Defects Discovered During This Work

Two properties of this build profile silently invalidate time-dependent tests. Both had to be corrected before Z-02 and Z-05 could be observed at all, and both are worth recording because they would equally mislead future work on this repository.

H-1 — `via_ir` eliminates repeated `TIMESTAMP/NUMBER` reads. The `cauldron` profile compiles with `via_ir = true`. Its optimiser treats the `TIMESTAMP` and `NUMBER` opcodes as loop-invariant and common-subexpression-eliminates them, so

```
| for (uint256 i = 0; i < 24; ++i) { vm.warp(block.timestamp + 150); engine.poke(); }
```

evaluates `block.timestamp` *once* and warps to the same target 24 times. Measured directly: 24 iterations advanced the clock by 150 seconds in total, and the engine’s `obsIndex` moved from 1 to 2 instead of to 25. Any test that advances a clock more than once per function must hold the clock in a local or storage variable.

H-2 — `vm.roll` re-snaps the timestamp on an unpinned fork. On a fork created without a pinned block, the block environment is re-synced behind the test, so `vm.roll` overwrites a previously warped `block.timestamp`. Pinning (`vm.createSelectFork(rpc, block.number - 64)`) makes the environment deterministic; even then, `vm.roll` must not follow a `vm.warp` whose elapsed time is under test.

The practical consequence for this codebase is worth stating plainly: under H-1, a test written to verify that the permissionless `poke()` keeps the perpetuals oracle fresh would appear to pass while never advancing the clock at all. That is precisely the shape of Z-02.